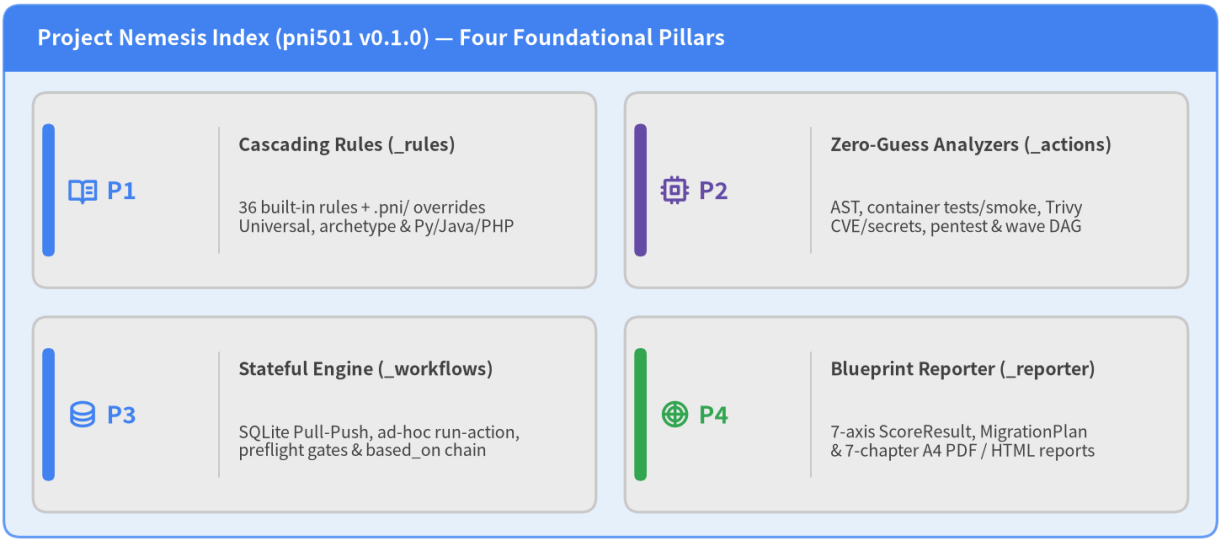
pni501 (Project Nemesis Index) — Architecture & Mechanism Specification

Package: `pni501` (`v0.1.0` | `import pni` / CLI: `pni`) | **Languages**: `Python` , `Java` , `PHP` | **Theme**: Google Style (`en`) | **Architecture**: AI-Native Deterministic Software Health & Migration Engine

Project Nemesis Index (`pni501`) is an AI-native software health evaluation, security auditing, and migration planning framework supporting `Python` , `Java` , and `PHP` repositories. By uniting **cascading multi-language engineering rules** (`_rules`), **zero-hallucination static, container, security, and penetration analyzers** (`_actions`), a **SQLite-backed stateful Pull-Push workflow engine** (`_workflows`), and **blueprint-driven Drawlib visual reporting** (`_reporter`), `pni501` eliminates LLM metric and path hallucinations while producing actionable 7-axis health scores, topological migration plans, and publication-grade 7-chapter A4 PDF reports.



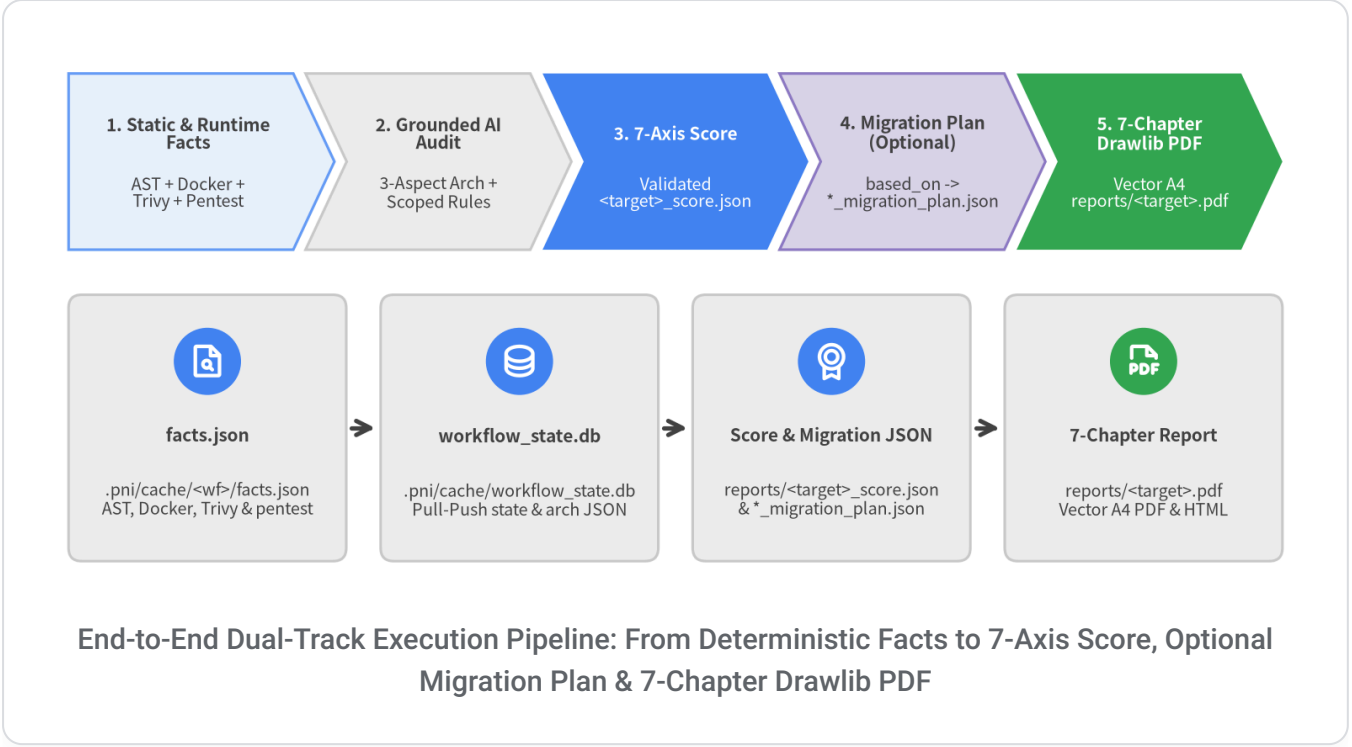
Core Architectural Pillars of pni501 (Project Nemesis Index v0.1.0)

Executive Overview: Hybrid Deterministic + Cognitive Auditing & Migration

Purely LLM-driven code reviews suffer from non-deterministic counting errors, invented module paths, skipped checklist categories, and complete blindness to runtime container behavior, dependency CVEs, and live HTTP attack surfaces. Conversely, purely static linters cannot evaluate domain boundary cohesion, aspect-separated system architecture, or phased modernization trade-offs.

`pni501` (`v0.1.0`) bridges both worlds across `Python` , `Java` , and `PHP` through `36` **built-in cascading rules** (`common/` , `python/` , `java/` , `php/`), `21` **reusable composite actions**, and `12` **canonical** `PrimitiveType` **s** orchestrated in a dual-track audit-to-migration pipeline:

- **Programmatic Multi-Modal Fact Extraction (`6` Deterministic Primitives)**: `pni` executes `project_inspect` , polyglot `ast_analyze` (Python `ast` + `ast-grep-py` for Java/PHP), isolated Docker `run_check` (unit tests, coverage, HTTP smoke checks), socketless Trivy `security_scan` (SCA CVEs, IaC misconfigurations, leaked secrets), loopback `pytest` + `httpx` `web_pentest` , and topological `migration_analyze` —persisting exact telemetry into `.pni/cache/<workflow_id>/facts.json` and `.pni/cache/workflow_state.db` .
- **Schema-Enforced Cognitive Audit, Chained Migration & Visual Reporting (`6` AI/Report Primitives)**: Guided step-by-step via `workflow_state.db` , the Code AI Agent loads scoped rules (`load_rules`), records path-verified 3-aspect architecture (`record_architecture` -> `architecture.json`), evaluates rule compliance (`ai_interpret`), emits a 7-axis `ScoreResult` (`reports/<target>_score.json`), optionally chains a `based_on` migration workflow into a validated `MigrationPlanResult` (`reports/<target>_migration_plan.json`), and compiles a 7-chapter **Drawlib visual report** (`reports/<target>.pdf` and optional HTML).



End-to-End Dual-Track Execution Pipeline: From Deterministic Facts to 7-Axis Score, Optional Migration Plan & 7-Chapter Drawlib PDF

Table of Contents

1. Background, Problem Statement & The PNI Approach

2. High-Cohesion Domain Architecture & Cascade Resolution

3. Two-Tier Action System & Deterministic Fact Extraction

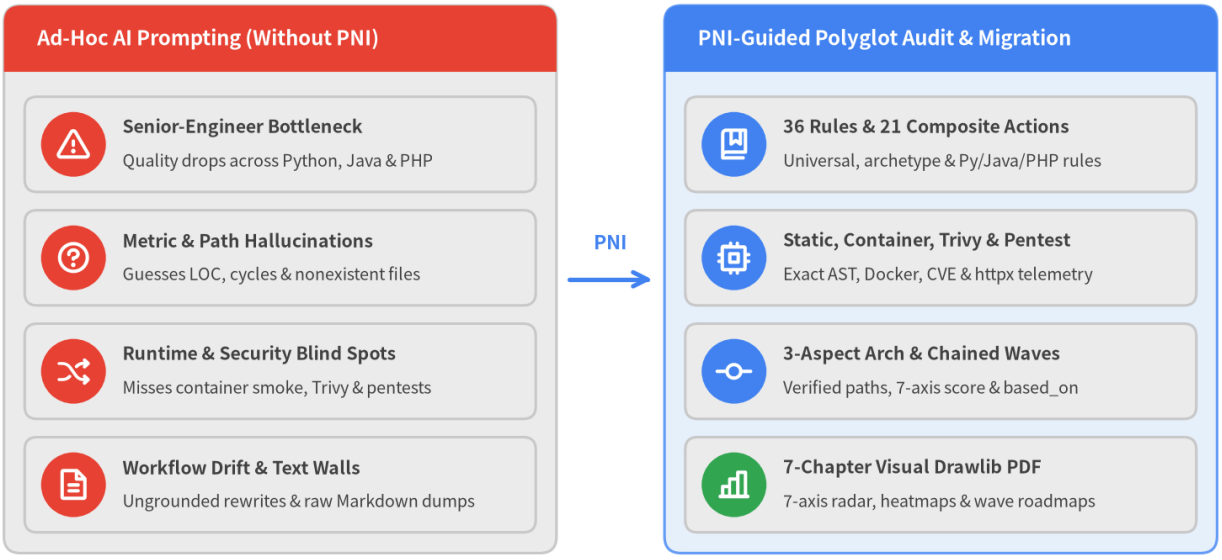
4. Workflow Usage Guide: Execution Modes, Pre-Flight Gates & YAML Expansion

5. Inside the Workflow Engine: Pull-Push Protocol, SQLite State & Migration Chaining

6. 7-Axis Scoring, Blueprint Reporter, Benchmarks & CLI Reference

1. Background, Problem Statement & The PNI Approach

Modernizing legacy and evolving software across `Python`, `Java`, and `PHP` remains one of the highest-leverage—and highest-risk—challenges in engineering organizations, yet applying Code AI Agents reliably still bottlenecks on senior human supervision unless polyglot standards, static and runtime telemetry, stateful workflows, and visual reporting are unified into a single deterministic harness.



The AI Modernization Gap: Ad-Hoc Prompting vs. Codified PNI Polyglot Audit & Migration Rigor

1.1 Background: Multi-Language Legacy Modernization in the Generative AI Era

Across enterprise backends, SaaS platforms, and internal developer tooling, **legacy modernization spans heterogeneous language ecosystems—most notably `Python`, `Java`, and `PHP`**. Engineering teams routinely inherit Python services with flat script layouts and untyped boundaries, Java Spring or servlet applications with tangled package dependencies

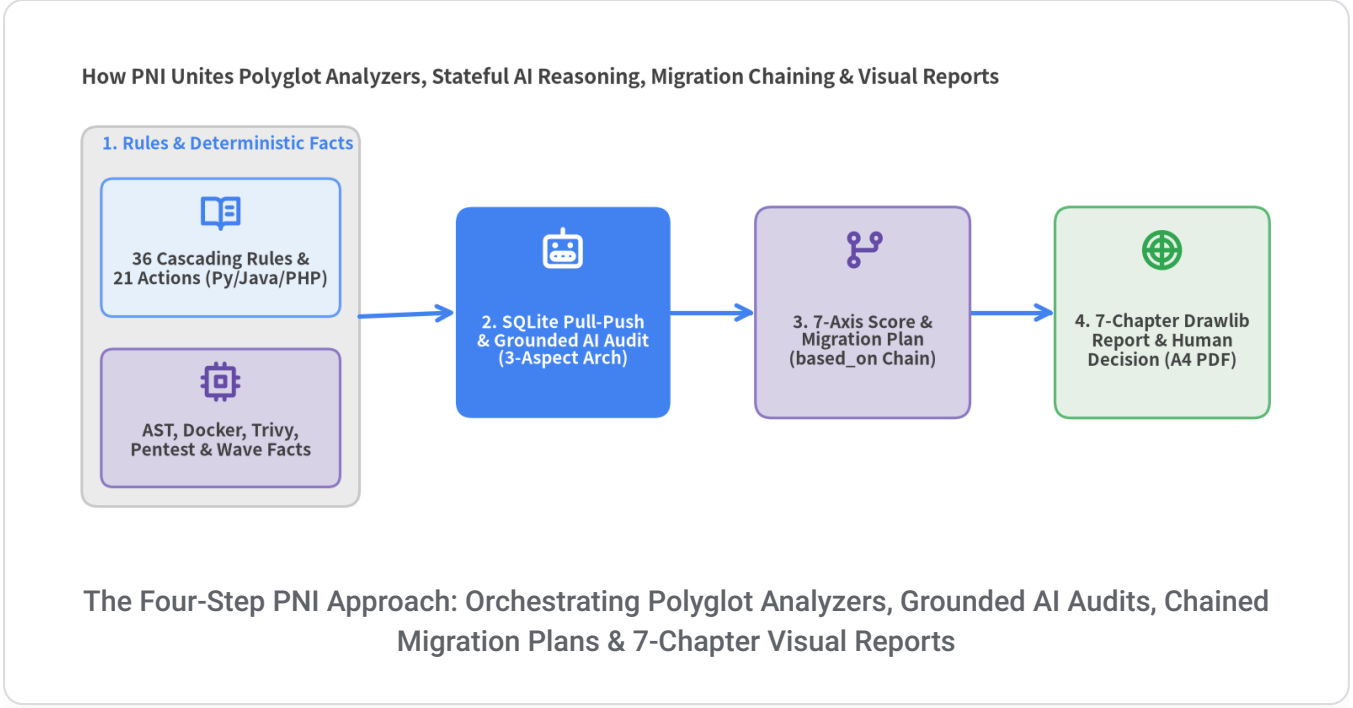
and aging Maven/Gradle trees, or PHP monoliths (`Laravel` , `WordPress` , or custom Composer/vanilla codebases) carrying raw SQL/template sinks and sparse automated tests. Before any team can safely refactor, re-architect, or migrate such repositories, engineers need an objective, reproducible baseline of structural health, runtime behavior, and security posture.

While **Code AI Agents** have dramatically accelerated code reading and transformation, unconstrained ad-hoc prompting suffers from four systemic failure modes:

1. **Senior-Engineer Prompt Bottleneck Across Ecosystems**: Crafting prompts that simultaneously enforce universal architecture principles (OWASP ASVS, OpenSSF supply-chain hygiene, Twelve-Factor readiness, CLI ergonomics) alongside language- and framework-specific idioms (`FastAPI` , `Spring Boot` , `Laravel` , `WordPress`) requires deep staff-level expertise on every run.
2. **Quantitative & Structural Hallucinations**: When asked to inspect a repository directly in chat, LLMs routinely guess LOC counts, fabricate type-annotation or generic coverage ratios, overlook cyclic dependency DAGs, and cite module paths that do not exist in the workspace.
3. **Runtime, Vulnerability & Penetration-Testing Blind Spots**: Static prompt-only reviews never verify whether tests actually pass inside a clean container, what line/branch coverage is achieved, whether `Trivy` flags critical CVEs, IaC misconfigurations, or hardcoded secrets, or whether live HTTP endpoints resist auth-bypass, SQLi/XSS, and SSRF probes under dynamic penetration testing.
4. **Workflow Drift, Ungrounded Migration Plans & Wall-of-Text Overload**: In long chat sessions, agents skip audit dimensions, propose risky "big-bang" rewrites disconnected from the real dependency graph, and dump thousands of lines of raw Markdown that overwhelm human reviewers.

1.2 The PNI Approach: Four Pillars of Deterministic AI Auditing & Migration

Project Nemesis Index (`pni501` `v0.1.0`) closes this gap by codifying senior engineering methodology into a deterministic CLI and SQLite state machine that guides any Code AI Agent (`Claude Code` , `Gemini CLI` , `Cursor` , `Antigravity` , etc.) from raw repository telemetry to a 7-axis health audit, an optional chained migration plan, and a 7-chapter visual report:



The Four-Step PNI Approach: Orchestrating Polyglot Analyzers, Grounded AI Audits, Chained Migration Plans & 7-Chapter Visual Reports

1. Cascading Multi-Language Rules, Actions & Workflows (`_rules` & `_actions`)

Instead of relying on ad-hoc prompts, `pni501` ships `36` **built-in engineering rules** across `common/`, `python/`, `java/`, and `php/`—covering universal architecture, security (`OWASP ASVS` + vulnerability/attack-surface standards), OpenSSF supply-chain governance, Twelve-Factor cloud-native readiness, CLI ergonomics, library hygiene, migration planning, and framework best practices (`FastAPI`, `Spring Boot`, `Laravel`, `WordPress`)—paired with `21` **composite actions** that expand into `12` **canonical** `PrimitiveType`**s**. Every built-in asset supports non-destructive `.pni/` local cascading overrides per repository.

2. Zero-Hallucination Static, Container, Security & Pentest Analyzers

Anything countable, executable, or structurally provable is measured programmatically and cached in `.pni/cache/<workflow_id>/facts.json`:

- **Polyglot Static & Manifest Inspection** (`project_inspect`, `ast_analyze`): Parses `pyproject.toml`, `pom.xml` / `build.gradle`, `composer.json`, lockfiles, CI/governance metadata, Python `ast`, and `ast-grep-py` syntax trees for `Java` and `PHP` to extract exact `module_inventory`, LOC, typing/generics ratios, cyclomatic complexity, API endpoints, security sinks, and cyclic import graphs.

- **Isolated Runtime, Security & Loopback Pentest Execution (`run_check`, `security_scan`, `web_pentest`)**: Executes unit suites, coverage collection, and HTTP smoke probes in isolated containers; runs socketless `Trivy` scans (`-- vuln,misconfig,secret`) with zero Docker-daemon exposure; and runs target-tailored `pytest` + `httpx` Web penetration tests against loopback containers (enforcing fail-closed preflight checks on `.pni/pentests/<target>/`).
- **Topological Migration Wave Analysis (`migration_analyze`)**: Computes strongly connected dependency cycles, external boundary touchpoints, and bottom-up leaf-to-entrypoint `refactoring_waves` directly from the verified import DAG.

3. Stateful Pull-Push Execution, 3-Aspect Grounded Architecture & `audit -> migration` Chaining (`_workflows`)

`pni workflows` drives the agent through a SQLite state machine (`.pni/cache/workflow_state.db`) via `start`, `get-action`, and `push-action` (alongside zero-prep `run-action` for single-step iteration):

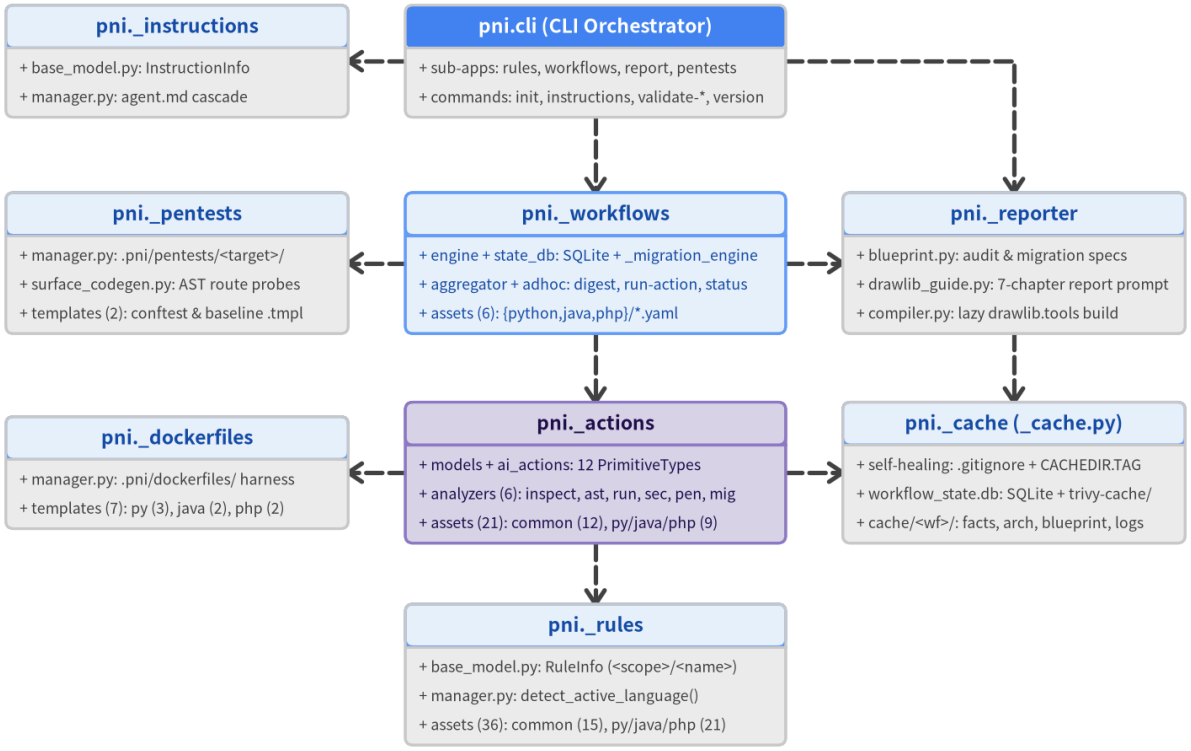
- **3-Aspect Grounded Architecture Recording (`record_architecture`)**: Before scoring, the agent records system comprehension across three explicit aspects—`purpose_and_stack`, `layers_and_modules`, and `flows_and_state`—while PNI validates every cited file and module path against `module_inventory` before persisting `.pni/cache/<workflow_id>/architecture.json`.
- **7-Axis Health Scoring (`emit_score`)**: The agent synthesizes rule compliance and prioritized findings into a schema-validated `ScoreResult` (`reports/<target>_score.json`) across **7 core engineering axes**: `architecture`, `code_quality`, `typing`, `testing`, `dependencies`, `security`, and `documentation`.
- **Prerequisite-Gated Migration Chaining (`based_on` -> `emit_migration_plan`)**: Migration workflows (`category: migration`) declare `based_on: <audit_workflow_id>`, failing closed unless a `COMPLETED` audit run exists in `workflow_state.db`, then inherit verified As-Is audit facts and architecture records to emit a path-validated `MigrationPlanResult` (`reports/<target>_migration_plan.json`).

4. Blueprint-Driven 7-Chapter Visual Drawlib Reports (`_reporter`)

To minimize human cognitive load during technical triage and architecture review, `_reporter` generates a deterministic JSON blueprint (`blueprint.json` or `migration_blueprint.json`) that guides the authoring of a **7-chapter Drawlib** `doc` **report** (`00_cover.md` + `01..06` under `reports_src/<target>_src/`). Combining executive `KpiCards` , 7-axis `RadarChart` s, 2D module `HeatmapChart` s, architectural risk `HeatmapOverlay` s, and phased `P0` – `P2` / wave `Roadmap` s, `pni report build` compiles crisp print-ready A4 vector PDFs (`reports/<target>.pdf`) and HTML deliverables with zero broken links.

2. High-Cohesion Domain Architecture & Cascade Resolution

`pni501` organizes its implementation under `src/pni/` around **seven vertical domain packages** (`_workflows` , `_actions` , `_reporter` , `_rules` , `_pentests` , `_dockerfiles` , and `_instructions`), a self-healing cache utility (`_cache.py`), and a thin Typer + Rich entrypoint (`cli.py`). Each domain colocates its Pydantic v2 schemas, resolution or execution logic, and built-in `.md` , `.yaml` , `.dockerfile` , or `.tmpl` assets—preserving a strict module-level Directed Acyclic Graph (`circular_dependencies = []`) across all `107` Python source files.



High-Cohesion Domain Package Hierarchy and Module-Level Dependency DAG in src/pni/

2.1 High-Cohesion Domain Package Design (`src/pni/`)

Traditional CLI codebases often slice code horizontally—scattering Pydantic models across `models/`, YAML/Markdown loaders across `config/`, subprocess runners across `executors/`, and static files across `templates/`. Modifying a single audit capability then forces synchronized edits across four disconnected directories and inflates context overhead for both human maintainers and AI coding agents.

`pni501` instead organizes `src/pni/` into **self-contained vertical domain packages**. Top-level `src/pni/__init__.py` exports only package identity constants (`LIB_NAME = "pni501"`, `LIB_VERSION = "0.1.0"`, and `__version__`), while every subsystem colocates its schemas, orchestration modules, and packaged templates inside its own bounded directory:

Package / Module	Core Python Modules	Key Schemas & Responsibilities	Packaged Built-in Assets
`cli.py`	`cli.py`	Typer root `app` + 4 sub-apps (`rules`, `workflows`, `report`, `pentests`) and root commands (`init`, `instructions`, `validate-score`, `validate-migration-plan`, `version`)	Rich CLI help epilog & bootstrap prom
`_workflows/`	`base_model.py`, `state_db.py`, `manager.py`, `engine.py`, `_migration_engine.py`, `aggregator.py`, `adhoc.py`, `inspector.py`	`WorkflowDefinition`, `WorkflowFactsBundle`, `WorkflowStateDB`, `CompiledAuditDigest`; stateful pull/push engine, ad-hoc `run-action`, and run history inspection	`6` starter workflows: `{python,java,php}/{full_audit,qui`
`_actions/`	`base_model.py`, `manager.py`, `ai_actions.py`, `project_inspect.py`, `_repo_governance.py`, plus `ast_analyze/`, `runtime_check/`, `security_scan/`, `pentest/`, `migration_analyze/`	`PrimitiveType` (`12`), `ScoreResult`, `ArchitectureRecord`, `MigrationPlanResult`, and 6 deterministic fact extractors (`ast_analyze/` houses `base/` + `python/`, `java/`, `php/` checkers)	`21` composite YAML actions: `common` `python/` (`3`), `java/` (`3`), `php/` (`3`)
`_reporter/`	`base_model.py`, `blueprint.py`, `migration_blueprint.py`, `drawlib_guide.py`, `compiler.py`	`ReportBlueprint`, `ReportScaffoldResult`, `ReportBuildResult`; pre-sorts audit/migration chapter blueprints and lazily imports `drawlib.tools` in `compiler.py`	7-chapter Drawlib `doc` scaffolding &

Package / Module	Core Python Modules	Key Schemas & Responsibilities	Packaged Built-in Assets
`_rules/`	`base_model.py`, `manager.py`	`RuleInfo`, `SUPPORTED_LANGUAGES = ("python", "java", "php")`, `detect_active_language()`, and `<scope>/<name>` cascade resolution	`36` Markdown files: `common/` (`15`), `p` `java/` (`6`), `php/` (`8`)
`_pentests/`	`manager.py`, `surface_codegen.py`	Materializes and validates target-scoped `pytest + httpx` Web penetration suites in `.pni/pentests/<target>/`, generating `test_target_surface.py` from `AstFacts.api_endpoints`	`2` probe templates: `conftest.py.tmp` `test_baseline_http.py.tmpl`
`_dockerfiles/`	`manager.py`	Resolves `.pni/dockerfiles/_templates/` overrides or built-in container harnesses into `.pni/dockerfiles/` alongside an isolated `.dockerignore`	`7` templates: `python/{pip,poetry,uv` `java/{gradle,maven}`, `php/{composer,raw}.dockerfile`
`_instructions/`	`base_model.py`, `manager.py`	`InstructionInfo`; resolves `.pni/instructions.md` overrides over built-in agent bootstrap instructions	`agent.md`
`_cache.py`	`_cache.py`	Self-healing `.pni/cache/` initializer (`ensure_cache_dir`, `ensure_cache_parent`) and workspace-safe path resolvers (`normalize_target_subpath`)	Auto-generated `.pni/cache/.gitigno` `CACHEDIR.TAG`

Colocating Python modules and static assets inside each package makes `BUILTIN_*_DIR = Path(__file__).resolve().parent` deterministic in editable installs and packaged wheels alike.

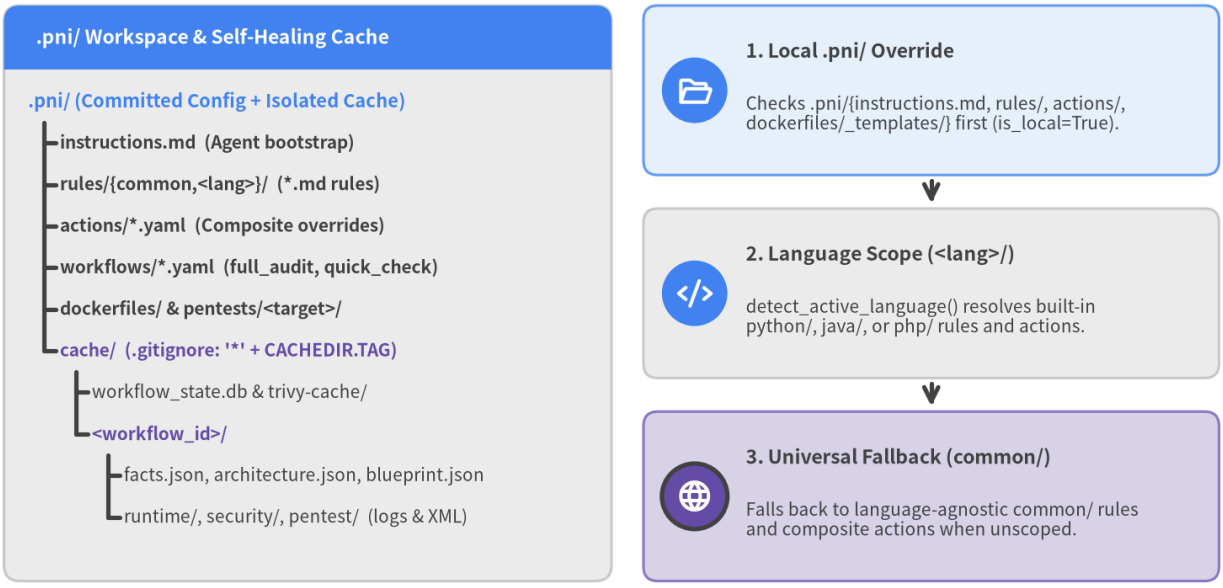
2.2 Module-Level Dependency Hierarchy & Lazy Tooling Isolation

Every import across `src/pni/` forms a strict **module-level Directed Acyclic Graph**—verified continuously by `pni501` running `ast_analyze` on its own `src/pni/` tree (`107` files, `circular_dependencies = []`):

1. **CLI Orchestrator (`src/pni/cli.py`)**: Parses user arguments, formats Rich tables and Markdown panels, and dispatches commands into `_workflows`, `_actions`, `_reporter`, `_rules`, `_pentests`, `_instructions`, and `_cache`. No internal module ever imports `pni.cli`.
2. **Stateful Workflow Orchestration (`src/pni/_workflows/`)**:
   - `engine.py` and `_migration_engine.py` coordinate SQLite persistence (`state_db.py`), preflight prerequisite gates (`based_on` audit completion and `.pni/pentests/` readiness), deterministic step execution, path-grounding verification, and `aggregator.py` score synthesis (`compile_workflow_digest`).
   - `adhoc.py` (`pni workflows run-action`) executes single workflow actions directly in memory with zero SQLite preparation, while `inspector.py` powers `pni workflows status` and `inspect-action` without exposing raw database queries.
3. **Action Subsystems & Supporting Harnesses (`_actions/`, `_reporter/`, `_pentests/`, `_dockerfiles/`)**:
   - Pure Pydantic data models in `_actions/base_model.py`, `_actions/ast_analyze/models.py`, and `_actions/migration_analyze/models.py` act as shared Leaf Schema contracts consumed cleanly by `_reporter/blueprint.py`, `_reporter/migration_blueprint.py`, and `_pentests/surface_codegen.py` without triggering cyclic runner imports.
   - Execution runners in `_actions/runtime_check/`, `_actions/security_scan/`, and `_actions/pentest/` delegate container harness setup to `_dockerfiles` and probe scaffolding to `_pentests`, while `_actions/ai_actions.py` delegates report prompt rendering to `_reporter/drawlib_guide.py`.
   - `_reporter/compiler.py` loads `drawlib.tools` **lazily** via `importlib.import_module("drawlib.tools")` behind a typed `_DrawlibToolsProtocol`, keeping `pni rules`, `pni workflows`, and `pni init` fast without importing Matplotlib/Cairo rendering stacks until `pni report` is invoked.
4. **Zero-Sibling Leaf Foundations (`_rules/`, `_instructions/`, `_cache.py`)**: Import zero sibling `pni._*` packages, providing pure language detection, Markdown cascade lookup, and path-traversal-guarded cache management across the workspace.

2.3 `.pni/` Workspace Layout & Three-Tier Cascade Resolution

Running `pni init [--lang/-l <python|java|php>] [--force/-f]` invokes `init_project()` in `src/pni/cli.py`, seeding a transparent `.pni/` configuration directory for the detected language while initializing the self-healing `.pni/cache/` hierarchy.



.pni/ Workspace & Self-Healing Cache Hierarchy Alongside Three-Tier Cascade Resolution

---

Self-Healing Cache & Target-Scoped Workspace Artifacts

Rather than modifying the host project's root `.gitignore`, `src/pni/_cache.py` (`ensure_cache_dir()` and `ensure_cache_parent()`) makes `.pni/cache/` **self-healing**:

- **Automatic Git & Backup Exclusion**: Every write into `.pni/cache/` automatically guarantees the existence of `.pni/cache/.gitignore` (`*`) and a standard `CACHEDIR.TAG` signature (`8a477f597d28d172789f06886806bc55`), keeping SQLite databases, Trivy vulnerability DB caches (`trivy-cache/`), JUnit XML reports, and container logs out of Git status and backup archives even in uninitialized repositories.
- **Workflow-Isolated Ephemeral State (`.pni/cache/<workflow_id>/`)**: Each workflow writes its aggregated snapshot to `facts.json`, grounded 3-aspect architecture model to

`architecture.json`, pre-sorted report specification to `blueprint.json` (or `migration_facts.json` and `migration_blueprint.json`), and raw stage artifacts into `runtime/`, `security/`, and `pentest/`.

- **Path-Guarded Multi-Target Harnesses (`.pni/dockerfiles/` & `.pni/pentests/`)**: `normalize_target_subpath()` rejects absolute paths and `..` traversal while mapping monorepo targets cleanly—for example, `targets.root: examples/php/legacy_shop_messy` maps to `.pni/dockerfiles/examples/php/legacy_shop_messy/Dockerfile` and `.pni/pentests/examples/php/legacy_shop_messy/`.

Three-Tier Cascade & Language Resolution Rules

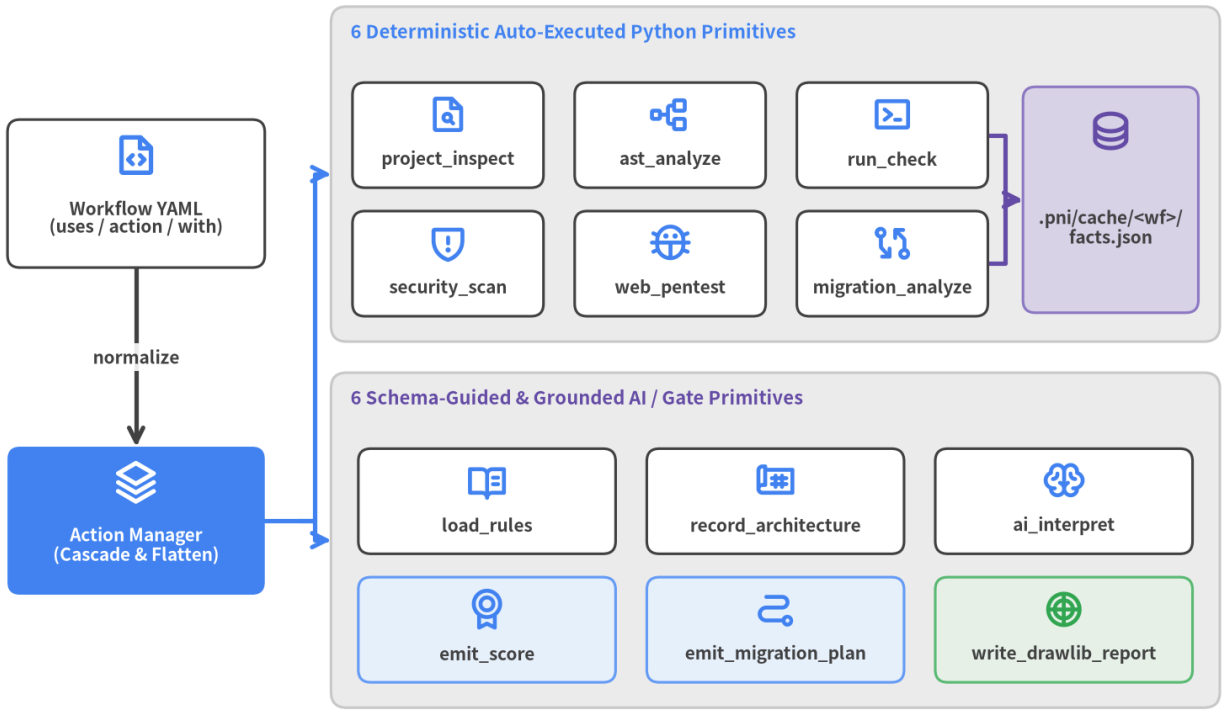
Teams can keep all seeded `.pni/` files (`37` files for Python, `36` for Java, `38` for PHP—excluding meta-guides `agent_instruction.md` and `workflow_guide.md` so `.pni/rules/` stays focused on project audit criteria) or delete unmodified files and retain only targeted overrides:

1. **Active Language Detection (`_rules/manager.py`)**: `detect_active_language(project_root)` inspects manifests in strict priority order—Python (`pyproject.toml`, `setup.py`, `requirements.txt`), Java (`pom.xml`, `build.gradle`, `build.gradle.kts`, `settings.gradle*`), PHP (`composer.json`, `wp-config*.php`, `artisan`, `phpunit.xml*`), TypeScript (`package.json`, `tsconfig.json`), and Go (`go.mod`)—before falling back to Java/PHP source-tree heuristics (`src/main/java`, `*.php`, `*.phtml`) and defaulting to `"python"`.
2. **Engineering Rules (`local > <lang> > common`)**: `list_rules()` merges `(scope, name)` candidates across built-in `src/pni/_rules/` and local `.pni/rules/` for `("common", active_lang)` (or all `SUPPORTED_LANGUAGES` with `--all`), preferring `.pni/rules/<scope>/<name>.md` (`is_local=True`). `get_rule_content()` resolves identifiers across hyphen/underscore normalization in four steps: exact `<scope>/<name>` -> `<active_lang>/<name>` -> `common/<name>` -> any matching rule name.
3. **Composite Actions (`local > <lang> > common`)**: `load_composite_action()` resolves `.pni/actions/<id>.yaml` (or scoped `.pni/actions/<scope>/<id>.yaml`) first, then built-in `<active_lang>/<id>.yaml` (enabling `uses: repo_structure_audit` to dispatch automatically to `python/`, `java/`, or `php/`), and finally `common/<id>.yaml`.
4. **Container Harnesses & Instructions**: `resolve_template_dockerfile()` prioritizes `.pni/dockerfiles/_templates/<lang>/<tool>.dockerfile` over built-in

`src/pni/_dockerfiles/<lang>/<tool>.dockerfile` (with per-language fallback to `pip`, `maven`, or `composer`), while `get_instructions_info()` prioritizes `.pni/instructions.md` over `src/pni/_instructions/agent.md`.

3. Two-Tier Action System & Deterministic Fact Extraction

To eliminate LLM guesswork over code metrics, dependency versions, runtime test pass rates, CVE counts, and refactoring blast radius, `pni501` decouples **declarative workflow authoring** from **execution mechanics**. Project workflows compose `21` language-aware YAML building blocks that flatten into `12` **canonical** `PrimitiveType`**s**—pairing `6` auto-executed Python analyzers with `6` schema-enforced, repository-grounded AI and gate primitives.



Two-Tier Action Expansion: 21 Cascading Composite Actions Flatten into 6 Deterministic Python Analyzers and 6 Grounded AI/Gate Primitives

3.1 Two-Tier Action Expansion (`21` Composite Actions -> `12` Primitives)

Workflow definitions in `.pni/workflows/*.yaml` (`WorkflowDefinition`) stay concise because `src/pni/_actions/base_model.py` separates reusable audit bundles from low-level execution steps:

1. **Tier 1 — Composite Actions (`CompositeActionDefinition`)**: Domain-focused YAML specifications invoked via `uses: <action_id>`. Each composite action bundles default `rules`, ordered `CompositePrimitiveSpec` entries, default `with` parameters, and domain-specific prompt instructions.
2. **Tier 2 — Canonical Primitives (`PrimitiveStep` & `PrimitiveType`)**: The `12` execution primitives referenced inside composite YAMLs or directly in a workflow step via `action: <primitive_type>`. Every stateful workflow run and ad-hoc `run-action` call operates over flattened `PrimitiveStep` instances.

Built-In Catalog of `21` Composite YAML Actions

`src/pni/_actions/` packages `21` **composite YAML actions** across `common/` (`12`) and `python/`, `java/`, `php/` (`3` each):

- `12` **Universal Actions (`common/`)**:
  - `architecture_analysis`: Expands into three sequential `record_architecture` steps (`aspect: purpose_and_stack`, `layers_and_modules`, `flows_and_state`) under `common/architecture`.
  - **Dynamic & Security Audits**: `container_runtime_audit` (`run_check` + `ai_interpret`), `security_vulnerability_audit` (`security_scan` + `ai_interpret`), and `web_penetration_audit` (`web_pentest` + `ai_interpret`).
  - **Archetype & Governance Audits**: `supply_chain_audit`, `cli_guidelines_audit`, `library_hygiene_audit`, and `service_readiness_audit` (specialized `ai_interpret` steps bound to OpenSSF supply-chain, CLI, library, 12-Factor, and SRE Production Readiness rules).
  - **Migration Planning**: `migration_proposal` (`migration_analyze` + `ai_interpret` + `emit_migration_plan` under `common/migration_planning`).

    - **Synthesis & Visual Reporting**: `emit_score` (`emit_score`), `write_drawlib_report` (`write_drawlib_report` with `report_kind: audit`), and `write_migration_report` (`write_drawlib_report` with `report_kind: migration`).
- `9` **Language-Scoped Actions (`python/`, `java/`, `php/`)**:
    - `repo_structure_audit` (one per language): Pairs `project_inspect` + `ai_interpret` with `<lang>/dependencies` (plus `php/modern_stack` for PHP) and universal supply-chain/documentation/Git/security rules.
    - `python_ast_audit` / `java_ast_audit` / `php_ast_audit`: Pairs `ast_analyze` + `ai_interpret` with `<lang>/code_quality`, `<lang>/typing`, modernization rules, and framework standards (`spring_boot`, `laravel`, `wordpress`).
    - `python_test_audit` / `java_test_audit` / `php_test_audit`: Evaluates test isolation, mirror layout, and assertion quality under `<lang>/testing`.

Cascade Flattening & Three-Layer Merging (`_actions/manager.py`)

When `normalize_workflow_steps()` (`src/pni/_workflows/manager.py`) expands a workflow, `expand_composite_action()` resolves `uses:` via `local (.pni/actions/) > <active_lang>/ > common/` and merges three configuration layers:

- **Parameters (`params`)**: Workflow-level target/output paths (`workflow_id`, `target_root`, `source_targets`, `test_targets`, `doc_targets`, `facts_path`, `score_path`, `output_dir`) merge with the workflow step's `with:` block, overriding primitive-level `with:` defaults (`{**prim.with_params, **override_params}`).
- **Rules (`rules`)**: Composite rules, workflow-step `rules:` additions, and primitive-level rules are concatenated and deduplicated in declaration order (`dict.fromkeys`).
- **Provenance IDs**: Workflow-step `prompt:` overrides append cleanly below base prompts, producing compound step IDs (`{step_id}__{idx}_{prim.type.value}`) with `source_action=comp.id` before SQLite assigns sequential action IDs (`act-01`, `act-02`, ...).

3.2 Six Deterministic Python Fact Extractors

Whenever `pni workflows get-action` (or `push-action`) reaches a deterministic primitive, `_execute_deterministic_step_if_needed()` runs the corresponding Python analyzer **once**, stores its JSON in `action_runs.deterministic_output_json` inside `.pni/cache/workflow_state.db`, and updates `.pni/cache/<workflow_id>/facts.json` (`WorkflowFactsBundle`).

1. Repository, Supply-Chain & Governance Inspector (`project_inspect` -> `ProjectFacts`)

Implemented in `src/pni/_actions/project_inspect.py` and `src/pni/_actions/_repo_governance.py`, `run_project_inspect()` extracts language-agnostic structural, dependency, documentation, and OpenSSF supply-chain signals without invoking external package managers:

- **Multi-Language Manifests & Lockfiles**: Parses `pyproject.toml` / `setup.py` / `requirements.txt` (Python), `pom.xml` / `build.gradle*` / Gradle version catalogs (Java), and `composer.json` / WordPress plugin & theme headers (PHP). Detects `lockfiles_found` (`uv.lock`, `poetry.lock`, `gradle.lockfile`, `composer.lock`, etc.), `unpinned_dependencies`, `detected_frameworks`, `runtime_types` (`["library"]`, `["cli"]`, `["cli", "library"]`, `["web_api"]`, `["worker"]`), `legacy_language_findings`, and `tooling_modernization_issues`.
- **OpenSSF Supply-Chain & Repo Governance (`_repo_governance.py`)**: Audits `has_license`, `has_security_policy` (`SECURITY.md`), `has_dependency_automation` (Dependabot / Renovate), GitHub Actions `ci_supply_chain_issues` (mutable `@main` / `@master` action refs, missing `permissions:` blocks, fat inline bash scripts), `committed_secret_files` (`.env`, `.pem`, `.key`, service-account JSONs), Diátaxis `readme_headings` / `docs_inventory`, and static `container_hygiene_issues` (`:latest` base tags, root `USER`).

2. Multi-Language CST/AST Static Analyzer (`ast_analyze` -> `AstFacts`)

Implemented in `src/pni/_actions/ast_analyze/`, `run_ast_analyze()` parses each target file **once** via `ast-grep-py` into an immutable `ParsedUnit` (`SgNode` CST) without importing or executing untrusted code, then runs **9 modular checkers** (`ast_analyze/base/` with concrete `python/`, `java/`, and `php/` implementations):

1. **Import Hygiene** (`imports.py`): `import_hygiene_coverage_pct`, `unsorted_imports`, `wildcard_imports`, and `late_imports`.
2. **Type Safety** (`typing.py`): `type_safety_coverage_pct`, missing parameter/return/field annotations (`type_safety_issues`), `Any`/raw-generic/`mixed` usages (`weak_type_usages`), and `unscoped_type_ignores`.
3. **API Documentation** (`docs.py`): `docstring_coverage_pct`, `missing_docstrings`, `empty_docstrings`, and parameter/return `doc_signature_mismatches` across PEP 257, Javadoc, and PHPDoc.
4. **Code Quality & Inventory** (`quality.py`): Exact `total_files`, `total_loc`, `total_classes`, `total_functions`, per-file `module_inventory` (`public_types`, `public_callables`), `complex_functions` (`> 50` lines, `> 7` branches, or cognitive complexity `> 15`), `excessive_parameters` (`> 6`), `global_state_usages`, and `bug_risk_idioms`.
5. **Dependencies & Boundaries** (`dependencies.py`): `internal_import_graph`, Tarjan SCC `circular_dependencies`, static-only `type_checking_cycles`, `unused_dependencies`, `undeclared_dependencies`, and architectural `boundary_violations`.
6. **Test Suite Health** (`testing.py`): `test_functions_count`, `module_test_coverage_pct`, `assertion_free_tests`, and `conftest.py` hook-aware `test_harness_notes`.
7. **Framework & Version Compatibility** (`frameworks.py`, `version_compat.py`): `framework_issues`, `modernization_hints`, and `language_version_mismatches` against declared runtime constraints.
8. **Operational & 12-Factor Signals** (`operational.py`): `missing_io_timeouts`, `scattered_env_access`, `unstructured_logging`, and AST-level `hardcoded_secrets`.
9. **Security Sinks & Attack Surface** (`security.py`): CWE-classified `dangerous_sinks` (`sql_injection`, `xss`, `command_injection`, `unsafe_deserialization`, `code_execution`, etc.) and extracted HTTP routes (`api_endpoints` with `http_method`, `path`, `handler`, and `auth_mechanisms`).

3. Isolated Container Runtime Verification (`run_check` -> `RuntimeFacts`)

Implemented in `src/pni/_actions/runtime_check/`, `run_runtime_check()` verifies dynamic buildability, test execution, coverage, and boot health inside an isolated Docker or Podman container (persisting `.pni/cache/<workflow_id>/runtime/runtime_facts.json`):

- **Harness Materialization & Image Build**: Selects the matching `.pni/dockerfiles/` template (`uv`, `poetry`, `pip`, `maven`, `gradle`, `composer`, `raw`) via `selector.py`, validates any project `Dockerfile`, and builds the test harness image.
- **Offline Unit Tests & Measured Coverage**: Executes the test suite with `--network=none` (with optional `pre_test_cmd` and `pytest_extra_args`) so external network calls fail deterministically, parses JUnit XML (`TestExecutionResult` + `FailedTestCase` excerpts), and extracts line/branch coverage (`CoverageResult` + `lowest_covered_modules`) from `pytest-cov` JSON, JaCoCo XML, or Clover XML.
- **Archetype-Aware Smoke Probe (`smoke.py`)**: Runs a container startup probe tailored to `runtime_types`—`library` (clean module import), `cli` (`--help` exit `0`), `cli+library` (combined `--help` + SDK import), or `web_api` (boot + loopback HTTP probe)—classifying failures into `BootFailureKind`.

4. Socketless Containerized Vulnerability & Secret Scan (`security_scan` -> `SecurityFacts`)

Implemented in `src/pni/_actions/security_scan/`, `run_security_scan()` executes Aqua **Trivy** (`ghcr.io/aquasecurity/trivy`) inside a socketless container mounting a shared vulnerability DB cache at `.pni/cache/trivy-cache/` (persisting `.pni/cache/<workflow_id>/security/security_facts.json`):

- **Dual-Surface Scanning**: Scans both the repository filesystem (`scan_filesystem`) and an exported `docker save` / `podman save` tarball of the resolved container image (`scan_resolved_image`) without mounting `/var/run/docker.sock`.
- **App vs. OS CVE Separation & Redacted Secrets**: Splits deduplicated CVEs into `app_vuln_counts` (language lockfiles/manifests, annotating `is_unpinned_specifier` range matches) and `os_vuln_counts` (base image OS packages), alongside IaC/Dockerfile `misconfigurations` and redacted `SecretFindingItem` leaks.

5. Live Loopback Web Penetration Testing (`web_pentest` -> `PentestFacts`)

Implemented in `src/pni/_actions/pentest/`, `run_web_pentest()` performs dynamic black-box HTTP security probing against web applications (persisting `.pni/cache/<workflow_id>/pentest/pentest_facts.json`):

- **Fail-Closed Explicit Probe Suite**: Workflows containing `web_pentest` fail closed at `pni workflows start` unless `.pni/pentests/<target>/` contains `conftest.py` and `test_*.py` (scaffolded via `pni pentests init`, which combines baseline HTTP header/CORS/method checks with AST-derived `test_target_surface.py` route probes).
- **Loopback Container Execution**: Boots the target web server bound strictly to host loopback (`127.0.0.1:<port>`), runs `pytest + httpx` against the live container (or an explicit `--base-url`), tears down the container cleanly, and parses JUnit XML failures into CWE-tagged `PentestFindingItem` records.

6. Graph- & Contract-Driven Migration Intelligence (`migration_analyze` -> `MigrationFacts`)

Implemented in `src/pni/_actions/migration_analyze/`, `run_migration_analyze()` powers `category: migration` workflows anchored to a completed `based_on` audit run (persisting `.pni/cache/<workflow_id>/migration_facts.json`):

- **Tarjan SCC Cycle Cuts & Topological Waves** `1..4` (`graph_waves.py`): Detects strongly connected components in `internal_import_graph`, recommends minimal `cycle_cuts` (`CycleCutSuggestion`), and partitions modules from leaf DTOs/utilities (`Wave 1`) through domain/services (`Waves 2..3`) to entrypoints (`Wave 4`).
- **Implicit Contracts, Touchpoints & `DEC-XX` Candidates**: Reconstructs untyped request/response keys (`implicit_contracts` in `contracts.py`) and `env_contracts`, locates `in_memory_state`, `persistence_db`, and `external_http_rpc` couplings (`boundary_touchpoints` in `touchpoints.py`), matches automated codemods (`ruff`, `openrewrite`, `rector` in `codemods.py`), and flags ambiguous legacy behaviors as `DEC-XX` `spec_vs_bug_candidates` (`ambiguities.py`).

3.3 Complete Catalog of the `12` Canonical Primitives & Grounding Gates

In `src/pni/_actions/ai_actions.py` and `src/pni/_workflows/engine.py`, every `get-action` prompt injects `get_expected_input_schema_json()`, and every `push-action` call validates both **Pydantic v2 structure** and **repository path grounding** (`_verify_grounded_paths()` rejects any file or module path absent from disk and `AstFacts.module_inventory`):

Primitive	Mode	Validated Push Schema	Grounding & Side Effects
project_inspect	Deterministic	DeterministicAckInput	Parses manifests, lockfiles & governance -> .pni/cache/<wf>/facts.json
ast_analyze	Deterministic	DeterministicAckInput	Runs 9 ast-grep-py checkers (Py/Java/PHP) -> .pni/cache/<wf>/facts.json
run_check	Deterministic	DeterministicAckInput	Materializes .pni/dockerfiles/, runs --network=none unit tests & coverage
security_scan	Deterministic	DeterministicAckInput	Runs socketless Trivy FS + image tar scan -> security_facts.json & facts.json
web_pentest	Deterministic	DeterministicAckInput	Boots 127.0.0.1 loopback container & runs .pni/pentests/<target>/ httpx probes
migration_analyze	Deterministic	DeterministicAckInput	Computes Waves 1..4, Tarjan SCC cuts, contracts & DEC-XX -> migration_facts.json
load_rules	AI / Gate	DeterministicAckInput	Resolves & injects cascaded Markdown engineering rules into prompt context
record_architecture	AI / Grounded	SystemProfile | StructureMap | ExecutionFlowMap	Verifies paths against module_inventory -> .pni/cache/<wf>/architecture.json
ai_interpret	AI / Grounded	InterpretationInput	Grounds findings/action_items & runs normalize_step_rule_checks() per rule
emit_score	AI + Aggregator	ScoreResult	Runs aggregator.py digest + metric stamp -> reports/<target>_score.json
emit_migration_plan	AI + Grounded	MigrationPlanResult	Grounds As-Is paths + attaches MigrationFacts -> reports/<target>_migration_plan.json
write_drawlib_report	AI + Compiler	ReportCompletionInput	Emits blueprint.json & verifies non-empty compiled PDF deliverable on disk

Complete Catalog of the 12 Canonical PrimitiveType Actions, Execution Modes, Push Schemas, and Grounding / Side Effects

Grounding & Synthesis Mechanics Across the Cognitive Primitives

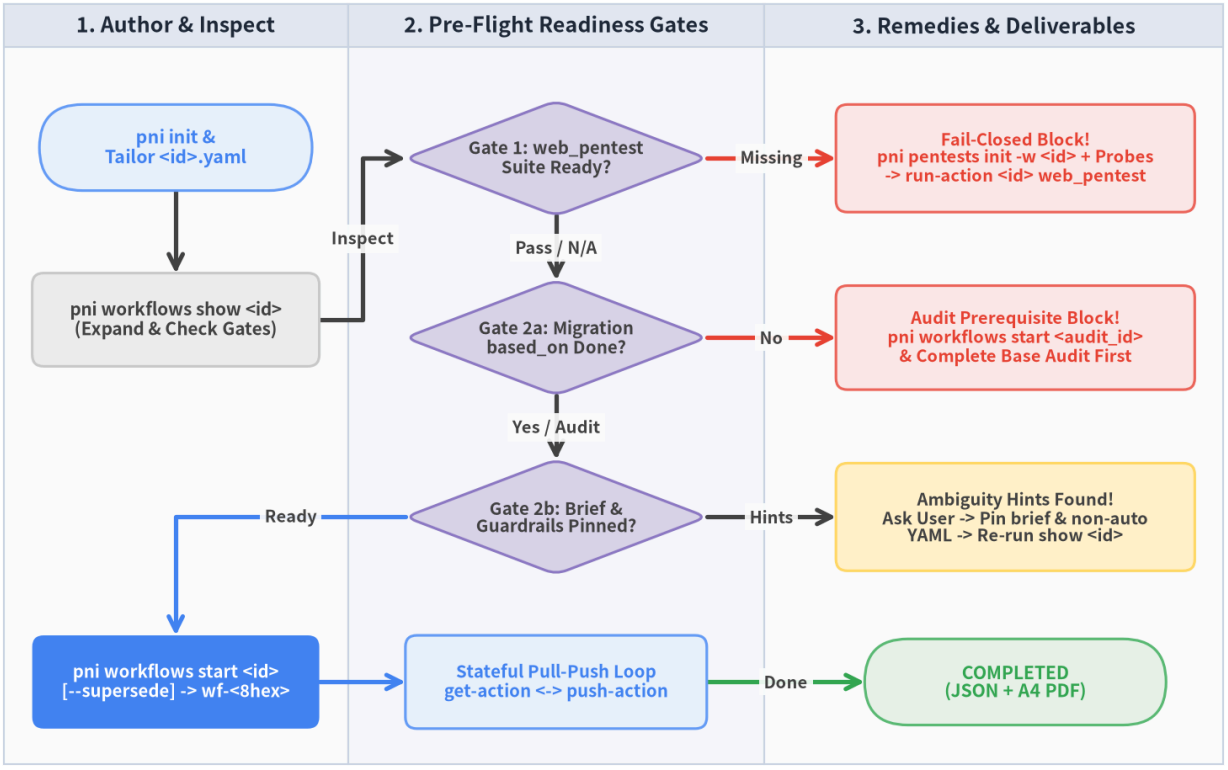
- **Aspect-Separated Architecture Comprehension (`record_architecture` -> `architecture.json`)**: Parameterized via `with: {aspect: purpose_and_stack |`

`layers_and_modules | flows_and_state}`, `record_architecture` splits codebase comprehension into three focused schemas (`SystemProfile`, `StructureMap`, `ExecutionFlowMap`). `_validate_and_persist_architecture()` checks every `layers[].paths`, `modules[].path`, `entry_points`, and `involved_files` entry against `AstFacts.module_inventory` and the workspace before persisting `.pni/cache/<workflow_id>/architecture.json`.

- **Per-Rule Compliance & Grounded Reasoning (`ai_interpret` -> `InterpretationInput`)**: Requires a `0..100` `step_score`, explicit `RuleCheckResult` verdicts (`pass`, `warn`, `fail`, `not_applicable`), grounded `findings`, and `P0` – `P2` `action_items`. On push, `engine.py` verifies every `file_path` and `target_files` entry and invokes `normalize_step_rule_checks()` (`aggregator.py`) so every active rule on the step is guaranteed a stamped compliance verdict.
- **Deterministic Digest & Score Stamping (`emit_score` -> `<target>_score.json`)**: `_enrich_score_result()` runs `compile_workflow_digest()` (`src/pni/_workflows/aggregator.py`) to deduplicate findings, group co-located violations into `consolidated_findings`, compute per-file `module_hotspots` (`0.0..1.0` risk density), derive canonical `S` – `D` `grade`, attach `RuntimeFacts` / `SecurityFacts` / `PentestFacts`, and overwrite `metrics_snapshot` directly from programmatic facts before writing `reports/<target>_score.json`.
- **Grounded Migration Plan Synthesis (`emit_migration_plan` -> `<target>_migration_plan.json`)**: `validate_and_persist_migration_plan()` (`_migration_engine.py`) validates `MigrationPlanResult` (`transformation_axes`, `target_architecture`, `component_mappings`, `phases`, `pr_backlog`, `decision_log`), verifies all referenced As-Is source paths exist in the repository, attaches deterministic `MigrationFacts`, and writes `reports/<target>_migration_plan.json`.
- **Blueprint-Guided Visual Report Gate (`write_drawlib_report` -> `ReportCompletionInput`)**: `render_drawlib_report_guide()` pre-computes `.pni/cache/<workflow_id>/blueprint.json` (or `migration_blueprint.json`) for the 7-chapter Drawlib `doc` report (`00_cover.md` .. `06_*.md`), and `_validate_report_completion()` verifies on disk that every file listed in `ReportCompletionInput.generated_files` exists inside the workspace with `st_size > 0`.

4. Workflow Usage Guide: Execution Modes, Pre-Flight Gates & YAML Expansion

Executing a reliable software audit or modernization plan requires more than launching a single command: directory targets must match the real repository layout, dynamic penetration probes must exist before container boot, and migration guardrails must be pinned deterministically before state is frozen into SQLite. `pni501` unifies this lifecycle under `pni workflows`, combining **fail-closed pre-flight readiness gates**, **three complementary operating modes**, and **automatic YAML target inheritance and composite step expansion**.



End-to-End pni workflows Operator & Agent Readiness Flowchart: Setup, Fail-Closed Pre-Flight Gates, and Execution

4.1 Operator & Agent Pre-Flight Readiness Workflow

Because `pni workflows start <id>` freezes resolved directory targets, action parameters, and rule bindings into `.pni/cache/workflow_state.db`, `.pni/workflows/<id>.yaml` serves as the **Single Source of Truth (SSOT)** for every run. Before starting a stateful workflow session, human operators and AI agents follow a four-step readiness sequence enforced by `pni workflows show <id>` and `start_workflow_run()` (`src/pni/_workflows/engine.py`):

1. **Bootstrap & Tailor** `.pni/workflows/<id>.yaml`:
    - Run `pni init` to scaffold `.pni/rules/`, `.pni/actions/`, `.pni/workflows/`, `.pni/instructions/`, and `.pni/cache/.gitignore` for the detected repository language (`python`, `java`, or `php`).
    - Inspect the repository's actual manifest and source layout (`src/`, `app/`, `src/main/java`, or monorepo subproject roots) and update `targets` (`root`, `source`, `tests`, `docs`) plus archetype/framework steps.
    - Run `pni workflows show <id>` to inspect resolved output paths, flattened primitive steps, and automated pre-flight readiness panels.
2. **Decision Gate 1 — Fail-Closed Web Penetration Test Check (`web_pentest`)**:
    - Whenever a workflow's normalized steps include `web_pentest` (typically via `uses: web_penetration_audit`) and `force_regenerate_pentests` is `false`, `inspect_pentests_prerequisite()` inspects `.pni/pentests/<target>/`.
    - **Fail-Closed Enforcement**: If `.pni/pentests/<target>/` is missing `conftest.py` or at least one `test_*.py` probe suite, `pni workflows show <id>` prints a red blocking panel and `pni workflows start <id>` raises `WorkflowPrerequisiteError` immediately—preventing half-baked runs that would stall mid-pipeline.
    - **Remedy Loop**: Materialize the target's `Python + httpx` suite with `pni pentests init -w <id>`, inspect AST-discovered routes and auth mechanisms to add concrete HTTP security probes in `.pni/pentests/<target>/test_*.py`, and iterate statelessly via `pni workflows run-action <id> web_pentest` until the suite executes cleanly.
3. **Decision Gate 2 — Migration Audit Prerequisite & Ambiguity Scrutiny (`category: migration`)**:
    - **Gate 2a — SQLite Base Audit Gate (`based_on`)**: Migration workflows must declare `category: migration` and `based_on: <audit_id>`. On `show` and `start`,

`inspect_base_audit_status()` queries `.pni/cache/workflow_state.db` for the latest `completed` run of `<audit_id>`. If `<audit_id>` has never run or its latest run is still `in_progress`, `start` blocks with an actionable error instructing the agent to start or resume `<audit_id>` and finish the As-Is audit first.

- **Gate 2b — Pre-Execution Brief & Guardrail Scrutiny**: `check_migration_preflight_hints()` inspects `MigrationOptions` and the merged `brief` / `brief_file` text, surfacing structured `PreflightReadinessHint` warnings whenever specs are underspecified:

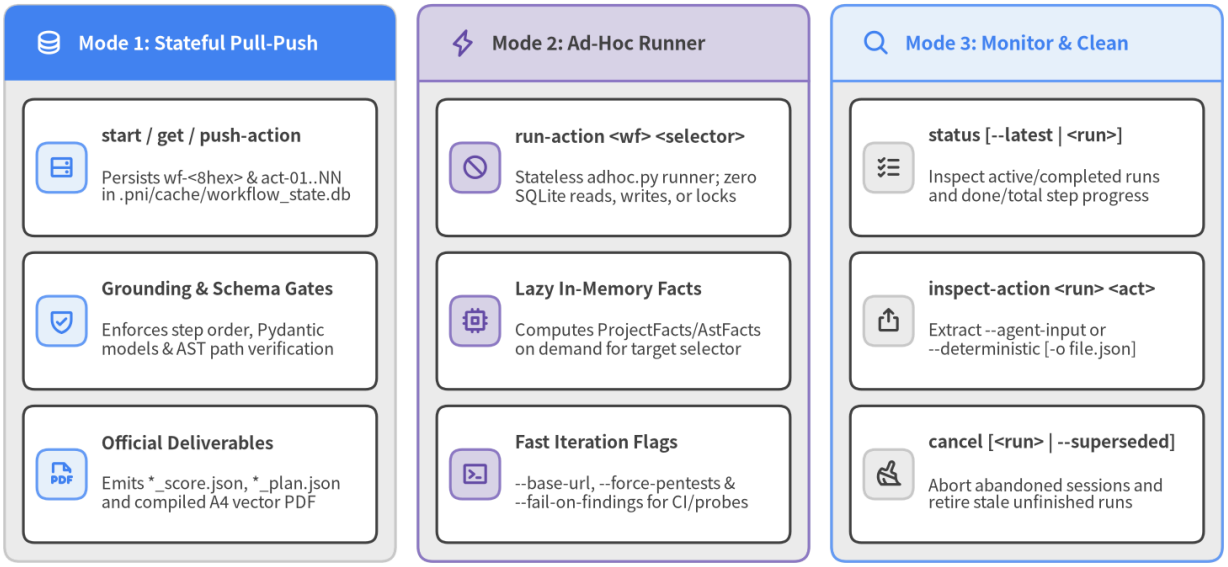
Hint Code	Trigger Condition	Required Operator / Agent Action Before `workflows start`
`MISSING_BRIEF`	Both `with.brief` and `with.brief_file` resolve to empty text	Cross-check As-Is audit facts, ask the user for target stack, boundaries, and compatibility, and write `with.brief: |` into YAML
`BRIEF_TOO_TERSE`	Resolved brief is shorter than `40` characters	Expand brief with concrete runtime/framework versions, DB/Cache backing stores, and explicit Non-Goals
`VAGUE_BRIEF_TERMS`	Brief contains placeholders such as `"to latest"`, `"TBD"`, `"TODO"`, `"undecided"`, `"最新"`, `"未定"`, or `"いい感じ"`	Replace vague placeholders with pinned target versions (e.g., `Java 21 LTS + Spring Boot 3.4`, `PHP 8.3 + Laravel 11`)
`UNPINNED_STRATEGY`	`migration_strategy` is left at `"auto"`	Confirm rollout model with user and pin `"phased_pr"`, `"strangler_fig"`, or `"rewrite"`
`UNPINNED_CODEMOD_TOOL`	`codemod_tool` is left at `"auto"`	Pin `"openrewrite"`, `"rector"`, `"ruff"`, or `"manual_only"` matching the confirmed target stack

- Following `common/migration_planning`, the agent cross-checks six ambiguity dimensions against the completed As-Is audit—**(1)** concrete runtime/framework versions, **(2)** stateful DB/Cache/Session middleware, **(3)** architecture ring scope (`domain` / `usecase` / `port` / `adapter`), **(4)** `api_compatibility` (`strict` vs `versioned_v2` / `flexible`), **(5)** `bug_handling_policy` (`ask_human` Stakeholder Decision Log vs `preserve_behavior` / `fix_in_phase3`), and **(6)** rollout strategy & explicit Non-Goals—then updates `.pni/workflows/<id>.yaml` and re-verifies `pni workflows show <id>`.

4. **Start & Execute Stateful Pull-Push Loop**:
   - Once all pre-flight checks report green, run `pni workflows start <id> --supersede` to obtain `wf-<8hex>` and iterate `get-action` <-> `push-action` through `COMPLETED`.

4.2 The Three Operating Modes of `pni workflows`

To support formal end-to-end evaluations, rapid single-step iteration, and non-invasive SQLite inspection without forcing agents to write raw `sqlite3` queries, `pni workflows` provides three distinct operating modes:



The Three Operating Modes of pni workflows: Stateful Pull-Push, Stateless Ad-Hoc Runner, and Run Monitoring & Inspection

Command & Flag Comparison Across Modes

Mode	Subcommands & Key Flags	touches `workflow_state.db`?	Primary Use Cases
Mode 1: Full Stateful Pull-Push (`engine.py`)	`start <wf> [--supersede]` `get-action <run_id>` `push-action <run_id> <act_id> [JSON | -f file.json]`	**Read / Write** (creates `wf-<8hex>` & `act-01..NN`, persists outputs & inputs)	Official repository audits, `based_on` chained migration plans, score/plan JSON emission, and A4 PDF report compilation
Mode 2: Ad-Hoc Single-Step Runner (`adhoc.py`)	`run-action <wf> <selector>` `[--base-url URL]` `[--force-pentests]` `[--fail-on-findings]`	**None** (100% stateless in-memory `WorkflowFactsBundle`)	Developing `.pni/pentests/<target>/` HTTP probes, tuning `.pni/dockerfiles/`, testing a single composite step, or CI gating

Mode	Subcommands & Key Flags	touches `workflow_state.db`?	Primary Use Cases
Mode 3: Monitoring, History & Cleanup (`inspector.py`, `state_db.py`)	`status [run_or_wf] [--latest] [-w wf] [-s status] [-n limit] [--json]` `inspect-action <run_or_wf> <action> [--agent-input | --deterministic | --json] [-o path]` `cancel [<run_id> | --superseded [-w wf]]`	**Read** (`status`, `inspect-action`) / **Status Update** (`cancel`)	Checking `done/total` progress, resuming interrupted sessions, exporting recorded JSON payloads to files, and cleaning up superseded runs

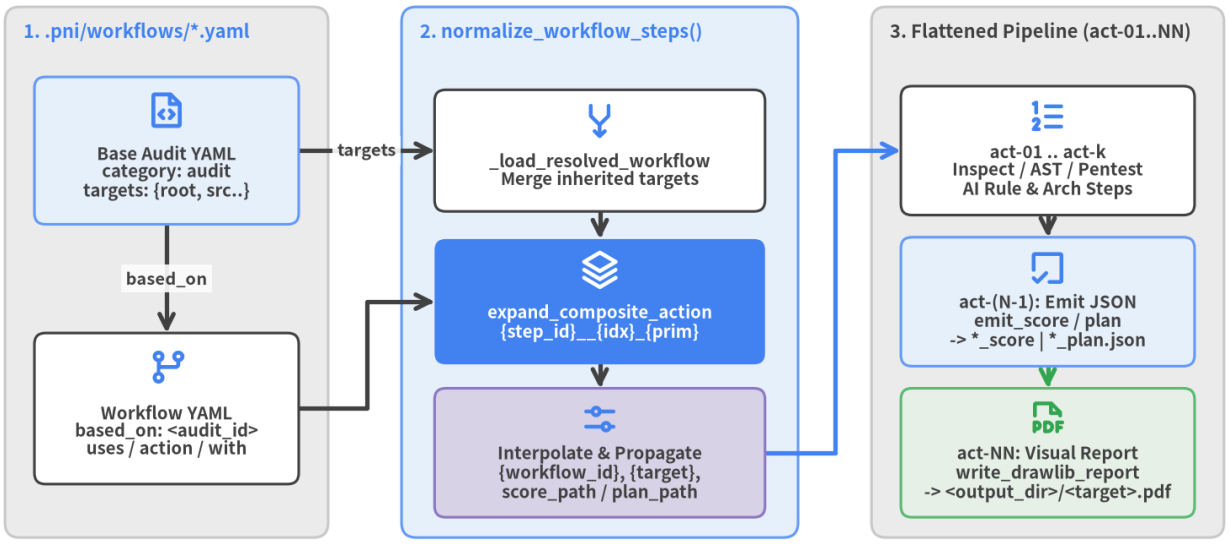
Stateless Lazy Evaluation in `run-action` (`src/pni/_workflows/adhoc.py`)

When an operator or agent runs `pni workflows run-action <workflow_id> <selector>`, `run_workflow_action_adhoc()` loads and normalizes `.pni/workflows/<workflow_id>.yaml` without creating a SQLite run or enforcing workflow-wide `based_on` prerequisites:

- **On-Demand Prerequisite Synthesis**: If the matched primitive requires upstream metadata—for example, `web_pentest` or `migration_analyze` needing `ProjectFacts` and `AstFacts`, or `run_check` / `security_scan` needing `ProjectFacts`—`_ensure_project_facts()` and `_ensure_ast_facts()` compute them lazily in memory and suppress silent helper outputs from the rendered Markdown via `_filter_display_bundle()`.
- **Paired AI Guidance Preview**: When `<selector>` matches a composite `uses:` step (such as `repo_structure_audit` or `web_penetration_audit`), `adhoc.py` executes the deterministic primitive(s) and renders both the extracted facts and the paired `ai_interpret` checklist, active engineering rules, and expected JSON Schema in one stdout pass.
- **Exit-Code Gating**: Execution or container boot errors (`status in {"error", "timeout"}`) always exit with code `1`; passing `--fail-on-findings` additionally exits with code `1` whenever unit test failures, Trivy vulnerabilities/misconfigurations/secrets, or live penetration findings are detected.

4.3 Workflow YAML Schema, Normalization & Selector Resolution

Workflow files live in `.pni/workflows/<id>.yaml` and validate against `WorkflowDefinition` (`src/pni/_workflows/base_model.py`). During loading and normalization (`src/pni/_workflows/manager.py`), declarative steps are transformed into flat `PrimitiveStep` sequences with deterministic parameter interpolation and cross-step output propagation:



Workflow YAML Inheritance, Composite Action Expansion, and Placeholder Interpolation Pipeline (normalize_workflow_steps)

Normalization Rules & Automatic Token Interpolation

1. **Category & `based_on` Invariants (`WorkflowDefinition`):**
   - `category: audit` (default) cannot declare `based_on` .
   - `category: migration` **must** declare `based_on: <audit_workflow_id>` referencing an existing `audit` workflow YAML (self-references and circular chains raise `WorkflowPrerequisiteError`).
   - `_merge_inherited_targets()` copies `root` , `source` , `tests` , and `docs` from the base audit workflow automatically while preserving any explicit child field overrides.

2. **Composite Expansion (`expand_composite_action`)**:
    - Steps declaring `uses: <composite_id>` resolve `<composite_id>.yaml` through the three-layer cascade (`.pni/actions/` > built-in `<lang>/` > built-in `common/`).
    - Each expanded primitive receives deterministic ID `{step_id}__{idx}_{primitive_type}` (for example, `repo_and_deps__1_project_inspect` and `repo_and_deps__2_ai_interpret`), deduplicated merged `rules`, merged `with` parameters (`step.with` overrides composite defaults), and `source_action = <composite_id>`.
    - Steps declaring `action: <primitive_type>` map 1-to-1 to a single `PrimitiveStep` with `id = step.id` and `source_action = None`.
3. **Placeholder Interpolation & Downstream Path Propagation**:
    - `_finalize_primitive_params()` injects workspace target context (`workflow_id`, `workflow_category`, `target_root`, `source_targets`, `test_targets`, `doc_targets`, `facts_path`) alongside canonical defaults (`src_dir: "reports_src"`, `output_dir: "reports"`, `target: "{workflow_id}"`, `score_path: "{output_dir}/{target}_score.json"`), then interpolates `{workflow_id}`, `{target}`, `{output_dir}`, and `{src_dir}` across all string parameters.
    - `_propagate_emitted_score()` forwards the resolved `score_path` from `emit_score` (or `plan_path` from `emit_migration_plan`) into subsequent `write_drawlib_report` steps automatically unless overridden explicitly on the report step.

Concise YAML Examples: Audit vs. Chained Migration Workflow

Example A — Service Audit Workflow (`category: audit` with `web_pentest`)

```yaml
id: order_api_audit
category: audit
name: Order API Health, Security & Penetration Audit
description: Static AST/supply-chain audit, 3-aspect architecture comprehension,
container/Trivy checks, and live HTTP pentest.

targets:
  root: "services/order_api"
  source: ["order_app"]
  tests: ["tests"]
  docs: ["README.md"]

steps:
  - id: arch_recording
    uses: architecture_analysis
  - id: repo_and_deps
    uses: repo_structure_audit
  - id: ast_and_quality
    uses: python_ast_audit
  - id: service_readiness
    uses: service_readiness_audit
  - id: runtime_verification
    uses: container_runtime_audit
  - id: security_audit
    uses: security_vulnerability_audit
  - id: web_pentest
    uses: web_penetration_audit
  - id: output_score
    uses: emit_score
    with:
      output_dir: "reports/python"
      target: "order_api"
  - id: create_report
    uses: write_drawlib_report
    with:
      src_dir: "reports_src/python"
      output_dir: "reports/python"
      target: "order_api"
      lang: "en"
      style: "google"
```

Example B — Chained Migration Workflow (`category: migration` with `based_on` & Pinned Brief)

```yaml
id: order_api_migration
category: migration
based_on: order_api_audit          # Inherits targets & requires COMPLETED run in workflow_state.db
name: Phased Migration Plan - Legacy Flask Order API -> Python 3.12 + FastAPI

steps:
  - id: migration_intelligence_and_plan
    uses: migration_proposal
    with:
      output_dir: "reports/python"
      target: "order_api_migration"
      migration_strategy: "phased_pr"     # Pinned (non-"auto"): phased_pr | strangler_fig | rewrite
      api_compatibility: "strict"         # Pinned: strict | versioned_v2 | flexible
      bug_handling_policy: "ask_human"    # Pinned: ask_human | preserve_behavior | fix_in_phase3
      codemod_tool: "ruff"                # Pinned: openrewrite | rector | ruff | manual_only
      brief: |
        Target Stack & Infrastructure (Confirmed):
          - Runtime & Web: Python 3.12 + FastAPI + Pydantic v2 DTOs (replace Flask untyped request.json)
          - Persistence & State: PostgreSQL 16 via SQLAlchemy 2.0 + Redis 7 TTL cache (eliminate module globals)
        Architecture & Compatibility Guardrails:
          - Clean Architecture rings (`domain` / `usecase` / `port` / `adapter`) with zero framework imports in `domain`.
          - Strict external HTTP route and JSON key backward compatibility; isolate Hyrum's Law quirks in DEC-XX log.
        Confirmed Non-Goals:
          - Do not split into separate microservices during this migration.

  - id: compile_migration_report
    uses: write_migration_report
    with:
      src_dir: "reports_src/python"
      output_dir: "reports/python"
      target: "order_api_migration"
      lang: "en"
      style: "google"
```

Action Selector Resolution Rules (`run-action` & `inspect-action`)

Both `pni workflows run-action <wf> <selector>` (`select_workflow_primitives` in `adhoc.py`) and `pni workflows inspect-action <run_or_wf> <selector>`

(`resolve_run_actions_by_selector` in `inspector.py`) accept flexible human- and agent-friendly selectors resolved in strict priority order:

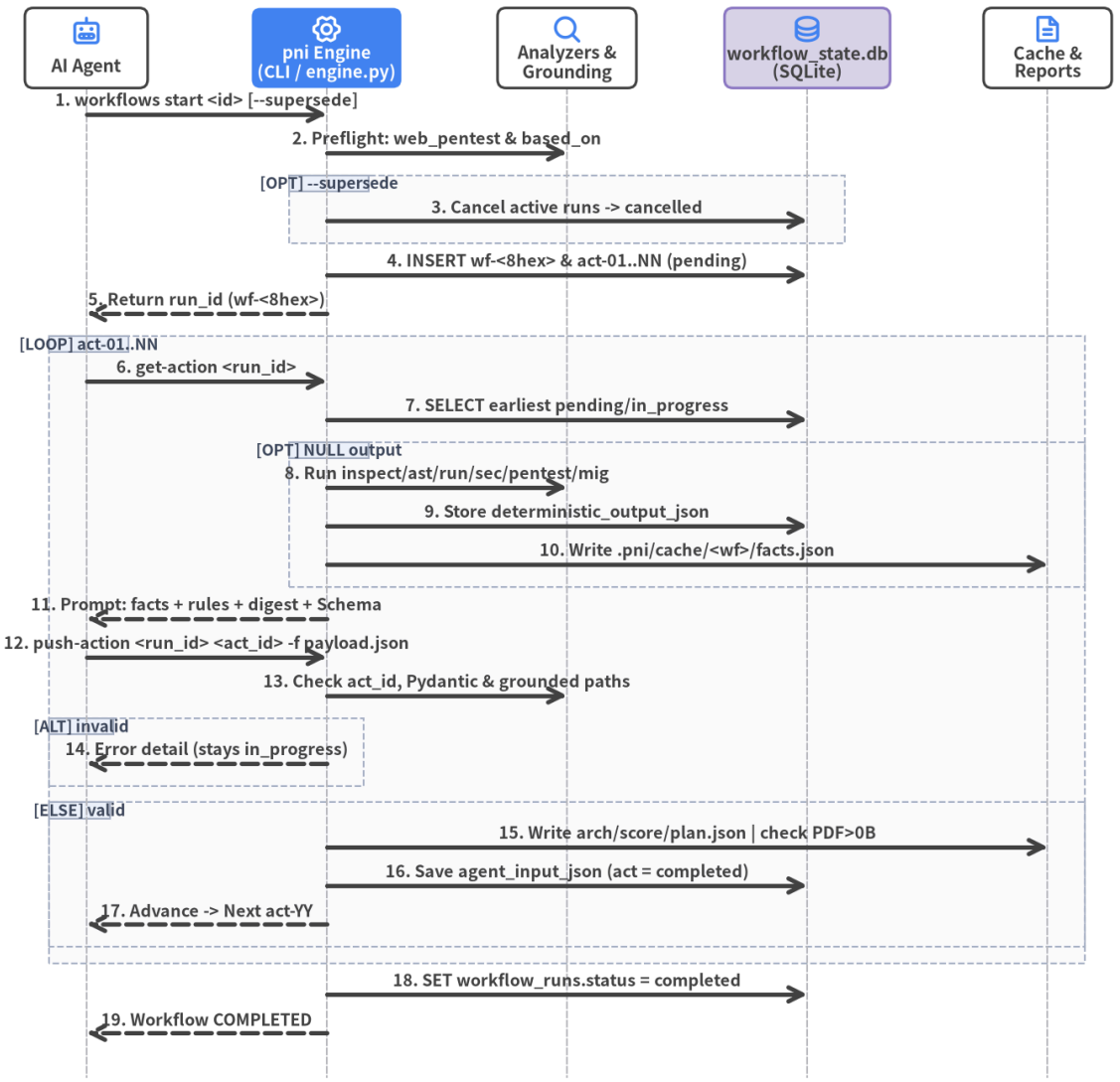
Priority	Selector Form	Example	`run-action` Match (`PrimitiveStep`)	`inspect-action` Match (`ActionRunRecord`)
1	**Step Index / `act-XX`**	`act-07` or `7`	1-based index into normalized `primitives`	Exact `seq == 7` (or exact `action_id == "act-07"`)
2	**Expanded Primitive ID**	`web_pentest__1_web_pentest`	Exact `prim.id == selector`	*(Stored as `act-XX` in SQLite; matched via index or step name)*
3	**Workflow Step ID**	`web_pentest` or `repo_and_deps`	All primitives whose `prim.id` prefix before `__` equals `selector`	Matched when `name` or `source_action` equals `selector`
4	**Composite `uses` ID**	`web_penetration_audit`	All primitives with `source_action` (or leaf after `/`) matching `selector`	All recorded actions with `source_action` (or leaf) matching `selector`
5	**Primitive Type**	`ast_analyze` or `emit_score`	All primitives where `prim.type.value == selector`	All recorded actions where `primitive_type.value == selector`

> **Multi-Step Narrowing in `inspect-action`** : When a composite selector such as `web_penetration_audit` matches both a deterministic step (`act-15: web_pentest`) and its paired AI step (`act-16: ai_interpret`), passing `--deterministic` automatically narrows the result to steps carrying `deterministic_output_json` , whereas passing `--agent-input` narrows the result to steps carrying substantive `agent_input_json` (`ai_interpret` , `record_architecture` , `emit_score` , `emit_migration_plan` , or `write_drawlib_report`).

5. Inside the Workflow Engine: Pull-Push Protocol, SQLite State & Migration Chaining

---

Once a workflow passes its pre-flight readiness gates, `pni501` transitions into a **deterministic stateful execution loop** orchestrated by `src/pni/_workflows/engine.py`, `state_db.py`, `aggregator.py`, and `_migration_engine.py`. Rather than trusting an AI agent to track multi-step progress in volatile chat memory or self-grade code paths without verification, the CLI acts as an authoritative state machine: executing static and container analyzers exactly once per step, synthesizing context-rich prompts on `get-action`, validating Pydantic schemas and repository path grounding on `push-action`, and chaining completed As-Is audit facts directly into downstream To-Be migration workflows.



Chronological Pull-Push Execution & Validation Sequence Across AI Agent, pni Engine, Grounding Analyzers, SQLite, and Cache

---

5.1 Chronological Pull-Push Protocol & Grounding Verification

Every stateful workflow run progresses through three CLI phases in `src/pni/_workflows/engine.py`:

1. Session Initialization (`start_workflow_run`)

1. Loads `.pni/workflows/<id>.yaml` with `based_on` target inheritance resolved.
2. Enforces both pre-flight gates (`inspect_base_audit_status` and `inspect_pentests_prerequisite`), raising `WorkflowPrerequisiteError` before touching SQLite if any prerequisite is unmet.
3. Normalizes composite steps into flat `PrimitiveStep` entries (`act-01` .. `act-NN`) and resolves effective `WorkflowOutputs` (`facts`, `score`, `report`).
4. When `--supersede` is passed, calls `db.cancel_active_runs_for_workflow(workflow_id)` to mark older unfinished runs and their remaining steps `cancelled`.
5. Inserts a new `workflow_runs` row (`run_id = wf-<8hex>`, `status = in_progress`, recording `based_on` and `based_on_run_id` when applicable) and `N` sequential `action_runs` rows (`status = pending`).

2. Deterministic Execution & Prompt Synthesis (`get_workflow_action`)

When the agent invokes `pni workflows get-action <run_id>`:

- **Idempotent Single-Shot Analyzer Execution**: `_execute_deterministic_step_if_needed()` checks `action.deterministic_output_json is None`. For deterministic primitives (`project_inspect`, `ast_analyze`, `run_check`, `security_scan`, `web_pentest`, and `migration_analyze`), the engine runs the analyzer **once**, stores the serialized Pydantic JSON in `action_runs.deterministic_output_json` alongside `status = in_progress`, and writes the merged `WorkflowFactsBundle` to `.pni/cache/<workflow_id>/facts.json`. Calling `get-action <run_id>` again after an interruption or validation retry re-renders the prompt instantaneously from SQLite without re-running Docker builds, Trivy scans, or HTTP probes.
- **Context-Complete Markdown Synthesis (`_format_action_guide`)**: The CLI assembles and prints:
    1. Step header (`run_id`, `action_id`, sequence `seq/total`, `primitive_type`, `source_action`, and `based_on_run_id`).
    2. Task-specific directive body (`render_action_task_body`), including per-aspect instructions on `record_architecture`, mandatory per-rule verdict checklists on

`ai_interpret`, and pre-computed `ReportBlueprint` / `MigrationReportBlueprint` layouts on `write_drawlib_report`.

3. Accumulated deterministic facts (`ProjectFacts`, `AstFacts`, `RuntimeFacts`, `SecurityFacts`, `PentestFacts`, `MigrationFacts`, `ArchitectureRecord`, and inherited `ScoreResult`).
4. Full Markdown bodies of active engineering rules (`_render_active_rules`).
5. Prior `ai_interpret` summaries and—immediately before `emit_score`—the deterministic `CompiledAuditDigest` (`compile_workflow_digest` in `aggregator.py`), which pre-deduplicates findings, computes module hotspot density, aggregates rule compliance matrices, and suggests weighted category scores.
6. Exact `push-action` CLI snippet and Pydantic JSON Schema (`get_expected_input_schema_json`).

3. Schema Validation, Grounding Gates & Side Effects (`push_workflow_action`)

When the agent submits `pni workflows push-action <run_id> <action_id> --file payload.json`:

1. **Strict Sequence Guard**: Verifies `action_id == current.action_id`; out-of-order pushes fail immediately without mutating SQLite.
2. **Pydantic Validation & Anti-Hallucination Grounding (`_verify_grounded_paths`)**: Every submitted payload is parsed against its canonical Pydantic v2 model and cross-checked against `AstFacts.module_inventory` and real workspace files (`_is_grounded_repo_path` strips `:symbol` / `#anchor` suffixes and prevents path traversal outside `project_root` / `target_root`):

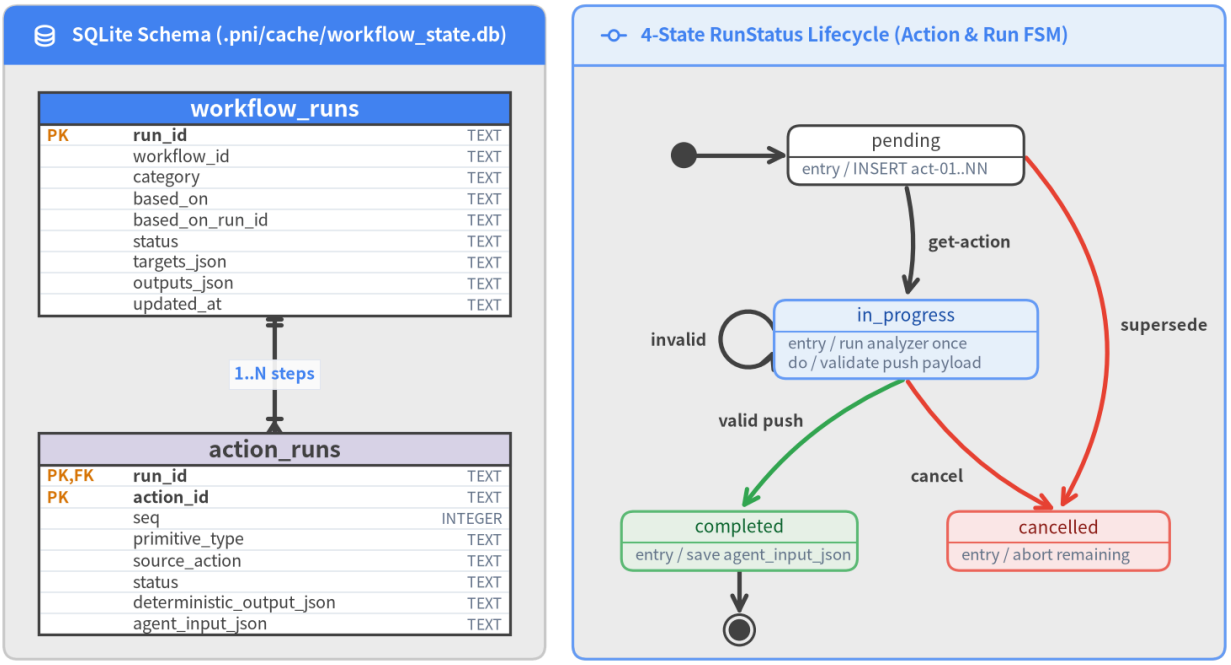
Primitive Type	Validated Pydantic Model	Grounding & Deliverable Side Effects on Valid Push
Deterministic (`project_inspect` .. `migration_analyze`)	`DeterministicAckInput` (`{"acknowledged": true}`)	Confirms analyzer output was reviewed and advances to the paired reasoning step
`record_architecture`	`SystemProfile` / `StructureMap` / `ExecutionFlowMap` / `ArchitectureRecord`	Verifies `layers[].paths`, `modules[].path`, `entry_points`, and `execution_flows[].involved_files`; writes `.pni/cache/<wf>/architecture.json` and updates `facts.json`

Primitive Type	Validated Pydantic Model	Grounding & Deliverable Side Effects on Valid Push
`ai_interpret`	`InterpretationInput`	Verifies `findings[].file_path` and `action_items[].target_files`; normalizes `rule_checks` against active step rules
`emit_score`	`ScoreResult`	Verifies finding/action paths; enriches `ScoreResult` with `CompiledAuditDigest`, `score_to_grade()`, and `metrics_snapshot`; writes `reports/<target>_score.json`
`emit_migration_plan`	`MigrationPlanResult`	Verifies `target_axes[].affected_files`, `component_mappings[].source_file`, `decision_log[].file_path`, and safe relative `pr_wbs[].target_files`; attaches `MigrationFacts` and writes `reports/<target>_migration_plan.json`
`write_drawlib_report`	`ReportCompletionInput`	Verifies every path in `generated_files` (e.g., `reports/<target>.pdf`) exists inside workspace and has `st_size > 0` bytes

3. **Self-Correction Without State Corruption**: If Pydantic parsing fails, a hallucinated file path is detected, or a PDF deliverable is missing/empty, `push_workflow_action()` returns `(False, error_detail)` while keeping the step in `in_progress`—allowing the AI agent to fix the JSON payload or re-compile the report and retry cleanly. Once valid, `complete_action()` stores `agent_input_json`, marks the step `completed`, and transitions `workflow_runs.status` to `completed` when the final step finishes.

5.2 SQLite Persistence Schema & 4-State `RunStatus` Lifecycle

All stateful workflow sessions are stored in `.pni/cache/workflow_state.db` (`WorkflowStateDB` in `src/pni/_workflows/state_db.py`, configured with `timeout=30.0` and automatic lightweight `ALTER TABLE` schema migrations for backward compatibility):



SQLite Persistence Schema (workflow_runs & action_runs) and 4-State RunStatus Lifecycle in .pni/cache/workflow_state.db

---

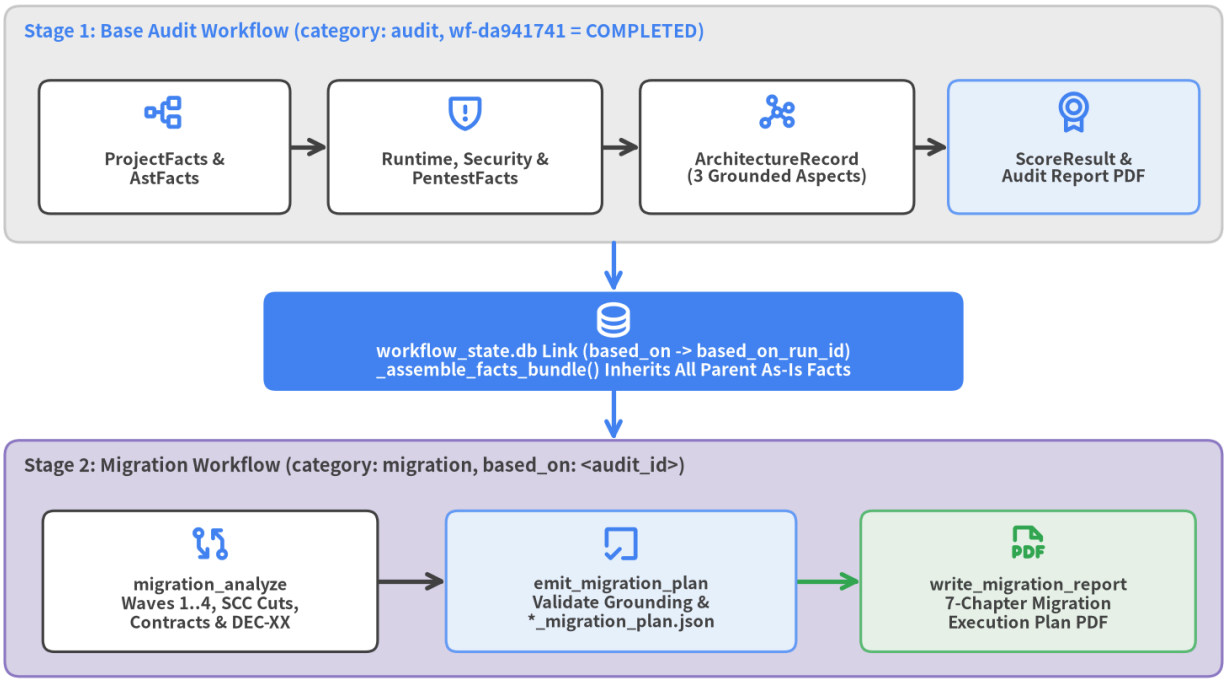
Key Table Responsibilities & Cancellation Semantics

- `workflow_runs`: Stores session-level provenance, including `run_id` (`wf-<8hex>`), `workflow_id`, `category` (`audit` or `migration`), `based_on` (parent workflow ID), `based_on_run_id` (the exact `wf-<8hex>` audit run pinned at `start` time), frozen `targets_json` (`WorkflowTargets`), and resolved `outputs_json` (`WorkflowOutputs`). New workflow runs start immediately in `in_progress`.
- `action_runs`: Stores the ordered `1..N` flattened primitive steps (`PRIMARY KEY (run_id, action_id)`), capturing `rules_json`, `params_json`, `prompt`, `deterministic_output_json` (populated on first `get-action`), and `agent_input_json` (populated on valid `push-action`).
- **Four-State Lifecycle (`RunStatus`)**:
  - `pending` -> `in_progress` -> `completed` forms the standard forward progression for every `action_runs` row.
  - Both `pending` and `in_progress` steps (along with their parent `in_progress` workflow run) can transition to `cancelled` via three mechanisms:

1. `pni workflows cancel <run_id>`: Explicitly aborts an abandoned `in_progress` session (`completed` runs are immutable and raise `ValueError` if cancellation is attempted).
2. `pni workflows cancel --superseded [-w <workflow_id>]`: Queries `workflow_runs` for unfinished runs that have been overtaken by a newer non-cancelled run of the same `workflow_id` and marks them `cancelled`.
3. `pni workflows start <workflow_id> --supersede`: Proactively cancels any active runs of `<workflow_id>` before inserting the new `wf-<8hex>` session.

5.3 Two-Stage Audit-to-Migration Workflow Chaining (`based_on` & `based_on_run_id`)

Large-scale modernization plans fail when an AI agent plans architectural refactoring from scratch without grounded As-Is diagnostics—or when every migration planning iteration has to re-build containers, re-run Trivy vulnerability scans, and re-execute live HTTP penetration probes. `pni501` solves both problems through **two-stage SQLite workflow chaining**:



Two-Stage Audit-to-Migration Workflow Chaining via based_on_run_id and _assemble_facts_bundle() in workflow_state.db

---

End-to-End Chaining Mechanics (`engine.py` & `_migration_engine.py`)

1. **Pinning `based_on_run_id` at Session Start**:
   - When `pni workflows start <migration_id>` executes, `inspect_base_audit_status()` locates the most recently `completed` run of `workflow_def.based_on` in `workflow_state.db` (e.g., `wf-da941741`) and records its exact identifier in `workflow_runs.based_on_run_id`.
   - Pinning `based_on_run_id` guarantees full reproducibility: even if someone later starts a new in-progress audit run, the active migration session remains bound to the exact completed audit snapshot it was validated against.
2. **Automatic Fact Bundle Reconstruction (`_assemble_facts_bundle`)**:
   - On every `get-action` and `push-action` inside Stage 2, `_assemble_facts_bundle(run, actions, db=db)` first extracts any facts produced in the current migration run, then queries `db.get_actions(run.based_on_run_id)` to hydrate:

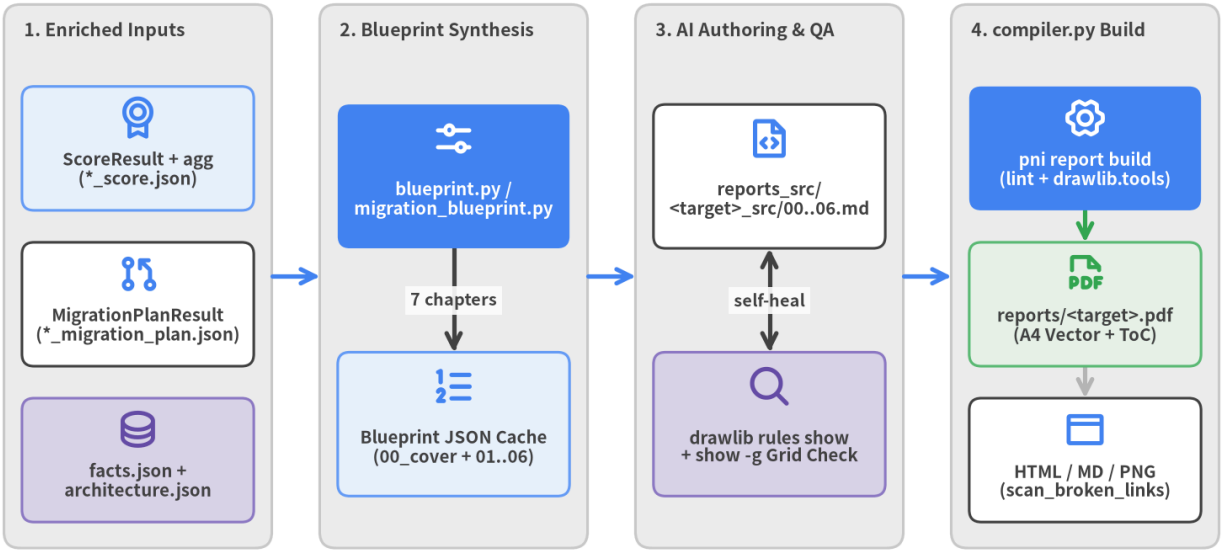
        - `bundle.project_facts`, `bundle.ast_facts`, `bundle.runtime_facts`, `bundle.security_facts`, and `bundle.pentest_facts` from the parent audit's `deterministic_output_json` columns.
        - `bundle.architecture_record` reconstructed across the parent audit's completed `record_architecture` steps (`purpose_and_stack`, `layers_and_modules`, `flows_and_state`).
        - `bundle.base_score_result` deserialized from the parent audit's `emit_score` `agent_input_json`, rendered directly into Stage 2 prompts as the **Baseline Audit Score Summary** (`overall_score`, category scores, and top inherited findings).
3. **Deterministic Migration Intelligence (`migration_analyze`)**:
    - Consuming the inherited `ProjectFacts`, `AstFacts`, and `RuntimeFacts` alongside the pinned YAML `MigrationOptions` (`brief`, `migration_strategy`, `api_compatibility`, `bug_handling_policy`, `codemod_tool`), `execute_migration_analyze_json()` computes `MigrationFacts`:
        - **Topological Refactoring Waves (`Waves 1..4`) & SCC Cycle Cuts**: Orders internal modules from zero-dependency leaves (`Wave 1`) up to ingress/orchestrators (`Wave 4`) and computes minimum-impact `cycle_cuts` across strongly connected components.
        - **Implicit Endpoint & Environment Contracts**: Extracts HTTP/CLI request and response keys (`implicit_contracts`) plus scattered environment variables and defaults (`env_contracts`) to seed Phase 0 characterization tests.
        - **Stateful, Database & HTTP Boundary Touchpoints**: Locates process-local mutable state, raw SQL calls, and outbound RPCs (`boundary_touchpoints`) paired with recommended Clean Architecture Port/Adapter targets.
        - **Mechanical Codemod Recipes & `DEC-XX` Ambiguity Candidates**: Matches stack-appropriate OpenRewrite, Rector, or Ruff recipes (`mechanical_codemods`) and isolates Hyrum's Law legacy quirks—such as loose equality auth checks, MD5 digests, unsafe cookie deserialization, or hidden debug keys—into numbered `DEC-01..NN` `spec_vs_bug_candidates`.
4. **Grounded Plan Persistence & 7-Chapter Visual Compilation**:
    - On `emit_migration_plan`, `validate_and_persist_migration_plan()` verifies that every As-Is path referenced in `target_axes[].affected_files`, `target_architecture.component_mappings[].source_file`, and

`decision_log[].file_path` exists in the inherited `AstFacts.module_inventory` or workspace, checks that `pr_wbs[].target_files` are safe relative paths, embeds `MigrationFacts`, and writes `reports/<target>_migration_plan.json`.

- Finally, `write_migration_report` (`write_drawlib_report` configured with `report_kind: "migration"`) generates `.pni/cache/<migration_id>/migration_blueprint.json` and guides the agent through compiling the 7-chapter (`00_cover.md` .. `06_*.md`) Drawlib Migration Execution Plan PDF (`reports/<target>.pdf`), verified non-empty before `workflow_runs.status` transitions to `completed`.

6. 7-Axis Scoring, Blueprint Reporter, Benchmarks & CLI Reference

Once PNI completes its deterministic static, container runtime, supply-chain security, and dynamic HTTP penetration analyzers alongside schema-enforced AI evaluations, it synthesizes raw diagnostic evidence into two executive-ready outputs: canonical JSON artifacts (`ScoreResult` and `MigrationPlanResult`) and multi-chapter **Illustration-as-Code** deliverables compiled through `drawlib`. Rather than asking an LLM to sort tables or invent layout structures from scratch, PNI deterministically enriches, deduplicates, and pre-sorts every metric into a 7-file `ReportBlueprint` before `compiler.py` renders vector A4 PDFs.



Blueprint-Driven Drawlib Report Pipeline Across Health Audit and Migration Execution Tracks

6.1 7-Axis `ScoreResult`, Deterministic `aggregator.py` & `MigrationPlanResult`

PNI enforces strict Pydantic v2 contracts on both terminal data artifacts—`ScoreResult` (`reports/<target>_score.json`) for health diagnostics and `MigrationPlanResult` (`reports/<target>_migration_plan.json`) for modernization execution plans—ensuring downstream automation, CI gates, and visual report compilers consume reproducible, path-verified records.

6.1.1 The 7 Core Engineering Axes & Grade Mapping

Every health audit evaluates a target project on a `0..100` integer scale across seven standardized engineering axes in `CategoryScores` (`src/pni/_actions/base_model.py`), accompanied by evidence-backed `CategoryRationales`:

1. `architecture`: Package/layer cohesion, dependency directionality, absence of strongly connected component (SCC) import cycles, and framework/domain boundary separation.
2. `code_quality`: SIG-aligned unit complexity, function length and parameter counts, idiomatic constructs, structured logging, I/O timeout hygiene, and absence of mutable global state.
3. `typing`: Type-annotation coverage (`type_safety_coverage_pct`), strictness declarations (`py.typed`, generics, `declare(strict_types=1)`), and static type-checker cleanliness.
4. `testing`: Automated test suite breadth, assertion density, containerized harness execution pass rate, smoke checks, and measured line/branch coverage.
5. `dependencies`: Manifest/lockfile determinism, version-pinning discipline, declared runtime compatibility (`language_version_mismatches`), and absence of undeclared imports.
6. `security`: Application and OS container CVE posture (Trivy), secret hygiene, IaC/Dockerfile hardening, CWE-classified AST sinks (`dangerous_sinks`), API authorization boundaries, and live HTTP penetration test verdicts (`web_pentest`).
7. `documentation`: Public module/callable docstring coverage (`docstring_coverage_pct`), onboarding/architecture docs, governance files (`SECURITY.md`, `LICENSE`), and operational runbooks.

Overall and per-step integer scores map deterministically to a five-tier `HealthGrade` via `score_to_grade()` in `src/pni/_workflows/aggregator.py`:

Grade	Score Threshold	Engineering Interpretation
`S`	`90 .. 100`	Exemplary reference architecture with near-zero structural, runtime, or security debt.
`A`	`80 .. 89`	Production-ready engineering hygiene with minor, non-blocking improvement opportunities.
`B`	`70 .. 79`	Sound structural foundation carrying localized test coverage, dependency, or hardening gaps.
`C`	`60 .. 69`	Elevated maintainability or operational risk requiring prioritized remediation before scaling.
`D`	`< 60`	Critical architectural, build/runtime, or exploitable security debt requiring structured intervention.

6.1.2 Deterministic Stamping (`_enrich_score_result`) & `aggregator.py` Pre-Sorting

When an agent submits `ScoreResult` during `emit_score`, `src/pni/_workflows/engine.py:_enrich_score_result()` never trusts raw LLM ordering or unverified metadata. Instead, it re-assembles the `WorkflowFactsBundle` from SQLite, verifies every referenced `file_path` and `target_files` entry against the real repository and `AstFacts.module_inventory` (`_verify_grounded_paths`), stamps immutable provenance (`workflow_id`, `workflow_run_id`, UTC `evaluated_at`, `target_root`, `facts_file`, recomputed `grade`, and deterministic `metrics_snapshot`), attaches `runtime_facts`, `security_facts`, `pentest_facts`, and `architecture_record`, and replaces list fields with the canonical output of `aggregator.py:compile_workflow_digest()`:

- `audit_dimensions` **(Weakest-First Step Scorecards)**: Aggregates every `ai_interpret` step into an `AuditDimensionScore` (`pass_count`, `warn_count`, `fail_count`, `score`, `grade`) sorted ascending by `(score, -fail_count, -warn_count, seq)` so the weakest engineering areas surface first. If an agent omits `step_score`, `_compute_step_score()` deterministically falls back to the mean of `suggested_scores` or `100 - sum(SEVERITY_DEDUCTION)` (`critical=35`, `high=20`, `medium=10`, `low=3`, `info=1`).
- `rule_compliance` (`fail -> warn -> pass -> not_applicable`): `normalize_step_rule_checks()` guarantees every active rule on every step receives a stamped `RuleCheckResult` (inferring verdicts from findings if omitted). `merge_and_sort_rule_compliance()` deduplicates rules across steps—retaining the

strictest status while concatenating unique evidence—and sorts by `(STATUS_RANK[status], rule_id)`.

- `module_hotspots` (`0.00 .. 1.00` **Risk Density Formula**): `compute_module_hotspots()` scores every source module and flagged infrastructure file on a bounded `[0.0, 1.0]` scale sorted descending by `(-risk_score, -loc, file_path)`:
  - Clean modules with zero findings and zero structural/runtime/security bonuses receive a baseline score of `0.06` (`_BASE_HEALTHY_RISK`).
  - Otherwise, `risk_score = round(min(1.0, raw_weight + ast_bonus + cov_bonus + sec_bonus), 2)`, combining:
    - **Finding severity weights** (`SEVERITY_RISK_WEIGHT`): `critical=+0.45`, `high=+0.28`, `medium=+0.15`, `low=+0.06`, `info=+0.02`.
    - **AST structural bonuses** (`_ast_structural_risk_bonus`): complex functions (`+0.12`), global state usages (`+0.14`), hardcoded secrets (`+0.25`), and dangerous sinks (`min(0.30, 0.12 * sink_count)`).
    - **Low measured test-coverage bonus** (`_build_low_coverage_bonus_map`): for modules with `>= 5` statements and `< 50%` measured line coverage, `+0.16` at `0.0%` coverage or `+0.10` at `0.1%..49.9%`.
    - **Trivy security bonus** (`_build_security_scan_bonus_map`): `+0.15` per secret or IaC misconfiguration hit on that file (capped at `+0.30`).
- `findings` & **Location-Grouped** `consolidated_findings`: `deduplicate_and_sort_findings()` sorts raw findings by `critical -> high -> medium -> low -> info`. Next, `consolidate_findings_by_location()` (`src/pni/_actions/base_model.py`) buckets overlapping violations sharing the exact same `(file_path, line)` into single multi-rule `ConsolidatedFinding` records (`rule_ids`, `combined_evidence`, `merged_recommendation`) sorted by `(severity_rank, file_path, line, rule_ids[0])`—eliminating duplicate table rows when multiple rules flag the same line of code.
- `action_plan` (`P0 -> P1 -> P2`): `deduplicate_and_sort_actions()` deduplicates remediation items by `(priority, title.lower())` and sorts deterministically by `(PRIORITY_RANK[priority], category, title)`.

6.1.3 `MigrationPlanResult` & Path-Grounded Modernization Schemas

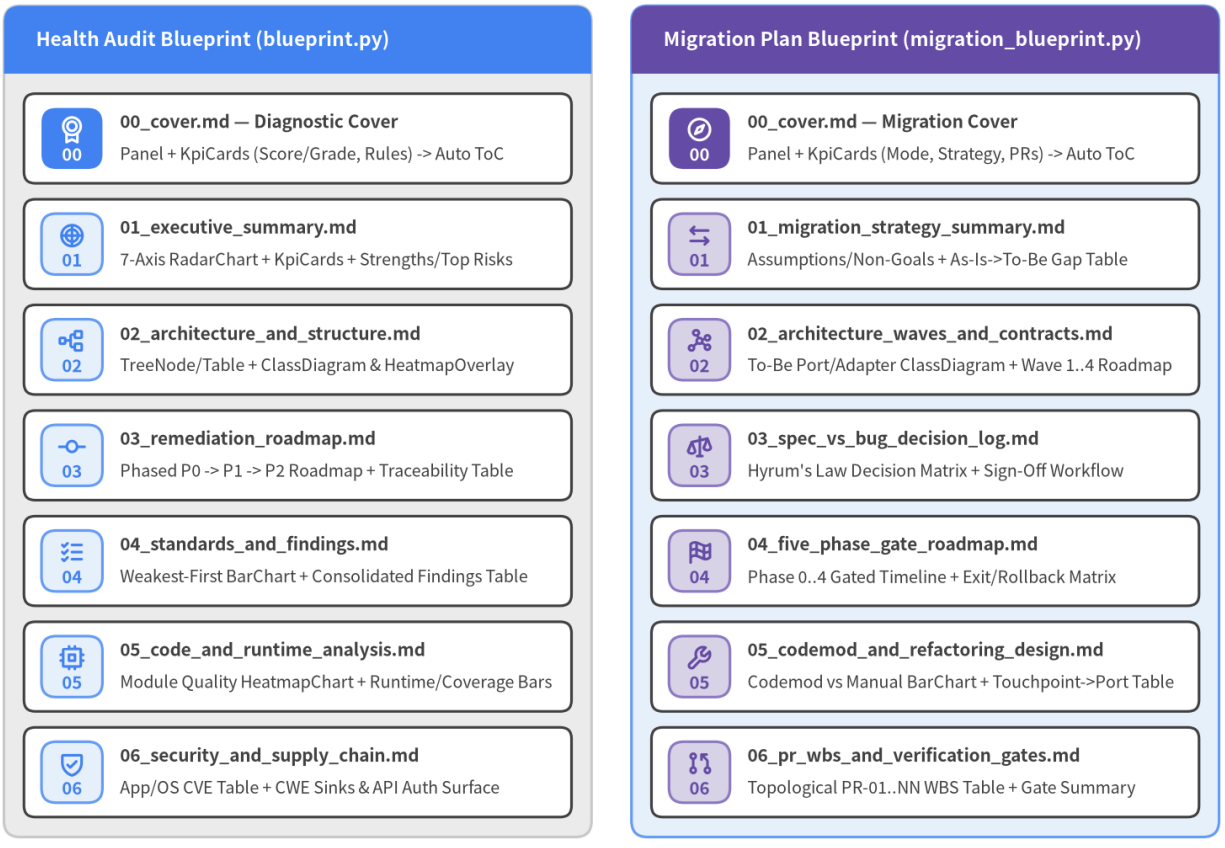
For repositories requiring multi-phase modernization (`category: migration` , `based_on: <audit_workflow_id>`), `emit_migration_plan` validates and persists `MigrationPlanResult` (`src/pni/_actions/migration_analyze/models.py`), verifiable via `pni validate-migration-plan <path>` :

- **Execution Guardrails**: Pins `transformation_mode` (`same_lang_upgrade` , `architectural_restructure` , or `cross_stack_replacement`), `migration_strategy` (`phased_pr` , `strangler_fig` , or `rewrite`), `api_compatibility` (`strict` , `versioned_v2` , or `flexible`), and `bug_handling_policy` (`ask_human` , `preserve_behavior` , or `fix_in_phase3`), alongside `confirmed_assumptions` and explicit `non_goals` .
- `target_axes` (`MigrationAxis`): Captures each As-Is $\to$ To-Be transformation vector (`language_runtime` , `framework` , `architecture_pattern` , `database_persistence` , `cache_state` , `auth_security` , `messaging_async` , `container_cicd` , `custom`) with `risk_level` and grounded `affected_files` .
- `target_architecture` (`TargetArchitectureDesign`): Defines the To-Be architecture (`pattern_name` , `dependency_rules` , `target_layers` with `allowed_dependencies` and `forbidden_imports`) and file-level `component_mappings` (`source_file` $\to$ `target_layer` , `target_component` , `port_or_adapter_role`).
- `decision_log` (`DecisionLogItem`): Isolates ambiguous legacy behaviors ("Spec vs. Bug" under Hyrum's Law, `DEC-01..NN`) with `current_behavior` , `impact_if_changed` , and `recommended_option` for Product Owner sign-off prior to Phase 3 business-logic changes.
- `phase_gates` (`MigrationPhaseGate` , **Phases** `0..4`): Enforces a 5-stage gated roadmap —**Phase 0** (Characterization & Harness Recovery), **Phase 1** (Cycle Cutting & Port Extraction), **Phase 2** (Mechanical Codemods & DB/Cache Adapters), **Phase 3** (Typed DTOs & Approved Fixes), and **Phase 4** (Cutover & CI/Container Hardening)—each with explicit `exit_criteria` and `rollback_or_coexistence_note` .
- `pr_wbs` (`MigrationPrTicket`) **& Attached** `migration_facts` : Topologically ordered pull-request tickets (`PR-01..NN` with `phase_number` , `wave 1..4` , `target_files` , `automated_or_manual` , `verification_cmd` , `estimated_size` , `depends_on_prs`), enriched automatically with deterministic `MigrationFacts` (`refactoring_waves` , `cycle_cuts` , `implicit_contracts` , `env_contracts` , `boundary_touchpoints` , `mechanical_codemods` , `manual_blockers` , and `spec_vs_bug_candidates`). During persistence, `_migration_engine.py:validate_and_persist_migration_plan()` verifies

that all As-Is paths exist in the repository while enforcing safe workspace-relative paths on To-Be `pr_wbs[].target_files`.

6.2 Dual 7-Chapter `ReportBlueprint` Architecture & Compiler Guardrails

During the final `write_drawlib_report` / `write_migration_report` step, `src/pni/_reporter/blueprint.py` and `src/pni/_reporter/migration_blueprint.py` convert the validated JSON artifacts into a cached `ReportBlueprint` (`.pni/cache/<workflow_id>/blueprint.json` or `migration_blueprint.json`). Both tracks standardize on an identical **7-file linear** `doc` **layout (**`00_cover.md` **+** `01..06`**)**, localized in English (`en`) or Japanese (`ja`), where Drawlib automatically injects a hyperlinked Table of Contents immediately after `00_cover.md`.



Side-by-Side 7-Chapter ReportBlueprint Architecture for Health Audits and Migration Execution Plans

6.2.1 Non-Destructive Scaffolding, On-Demand Rules & Compiler Preflight Checks

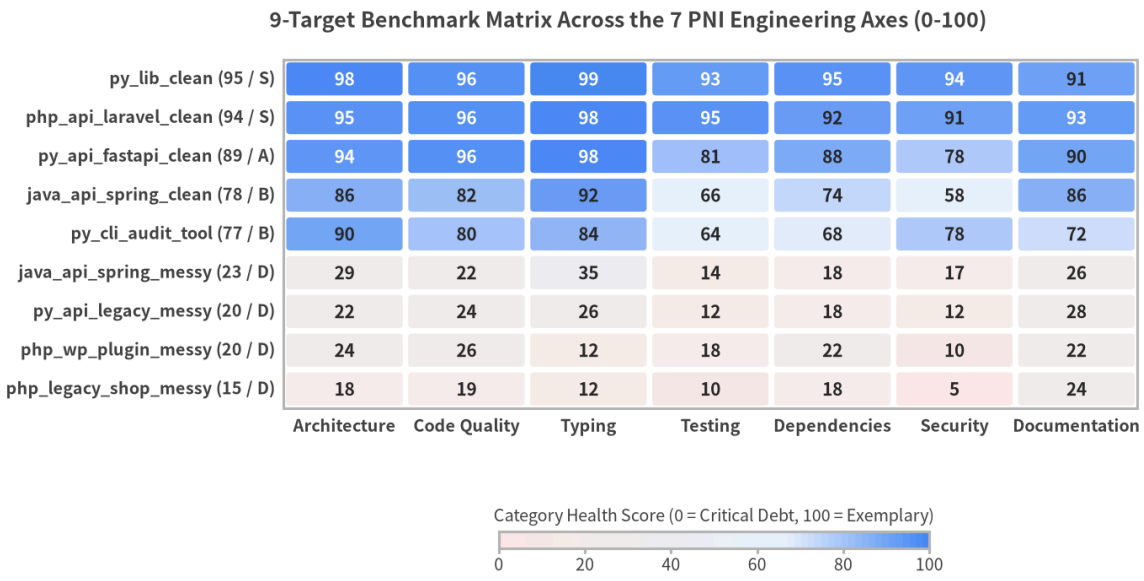
`src/pni/_reporter/compiler.py` and `drawlib_guide.py` enforce four engineering guardrails around visual report authoring and compilation:

1. **Non-Destructive `pni report init` (`scaffold_report_project`)**: Initializes `reports_src/<target>_src/` via `drawlib.tools.init_project("doc", ...)` with `drawlib.toml` and `<lang>/` localized chapter directories, and removes starter sample files (`<lang>/00_cover.md`, `01_overview.md`, `02_design.md`) on initial creation. If authored `<lang>/*.md` chapters already exist, `pni report init` preserves them intact—even when `--force` refreshes `drawlib.toml`, `styles.py`, `utils.py`, `style.css`, and `template.html`—so incremental re-audits produce clean `git diff` histories.
2. **On-Demand `drawlib rules show` Context Loading**: Rather than bloating early AST and rule-interpretation prompts with drawing documentation, PNI instructs agents to query `drawlib rules show` (`overview`, `project-doc`, `api`, `lib-smartarts-overview`, `lib-charts-overview`, `lib-diagrams-overview`, `lib-icons-overview`) right before authoring `reports_src/<target>_src/<lang>/*.md`, preventing guessed constructor arguments or coordinate-anchor inversions (`Bottom-Left` for Charts/Diagrams/Panels vs. `Top-Left` for `Table` / `TreeNode` vs. `Center` for primitives).
3. **Preflight Visual Linter (`lint_drawlib_chapters`)**: Before invoking `drawlib.tools.build_pdf()`, `compiler.py` scans every ```` ```drawlib ```` code fence across all chapter files to catch two recurring visual defects:
    - **Missing Multi-Series Legends**: Any `BarChart` or `LineChart` calling `.add_series(...)` two or more times must call `.draw_legend(...)`.
    - **Contour Label Collisions on UML Diagrams**: Combining `ClassDiagram` with `HeatmapOverlay(..., show_contour_lines=True)` emits a preflight warning (or raises `ValueError` under `strict_preflight=True`); authors must set `show_contour_lines=False` so thermal risk spots highlight high-risk classes cleanly without contour lines slicing through method signatures.
4. **Stage 2 Grid Verification (`-g`) & Multi-Format Link Scanning**: Agents export individual figures with the 10-unit/5-unit Cartesian grid overlaid (`uv run drawlib show ... -g -o .drawlib/scratch/<unique>_grid.png`) and inspect the rendered PNG multimodally to eliminate text clipping or box collisions prior to running `pni report build`. By default, `pni report build` compiles `reports/<target>.pdf` (`page_break=True`, `generate_index=True`, normalizing `<target>_<default_lang>.pdf` to `<target>.pdf`);

when `-f html` is requested, `_verify_links()` automatically runs `drawlib.tools.scan_broken_links()` and fails closed (`strict_links=True`) on any broken asset or anchor reference.

6.3 9-Target Multi-Language Benchmark Matrix & Reference Migration Plans

To validate diagnostic fidelity across clean production architectures, CLI tooling, and severely degraded legacy codebases, `pni501` maintains **nine end-to-end benchmark repositories** under `examples/{python,java,php}/` along with their committed `ScoreResult` JSONs and compiled A4 PDF reports under `reports/{python,java,php}/`.



Ground-Truth 7-Axis Health Scores Across All 9 Python, Java, and PHP Benchmark Targets

6.3.1 Multi-Language Health Audit Benchmark Summary

The nine benchmark targets span three clean reference implementations, two realistic mid-tier Grade `B` repositories (where PNI surfaces subtle container base-image CVEs, unauthenticated Actuator/API routes, or branch-coverage gaps despite clean code structure), and four severely degraded Grade `D` legacy applications:

Workflow ID	Project Name	Stack Profile	Overall	Arch	Qual	Type	Test	Deps	Sec
`py_lib_clean`	`metrics-kit`	Python 3.12 library	**95 (S)**	98	96	99	93	95	94
`php_api_laravel_clean`	`order-fulfillment`	PHP 8.3 / Laravel 11 API	**94 (S)**	95	96	98	95	92	91
`py_api_fastapi_clean`	`order-dispatch`	Python 3.12 / FastAPI	**89 (A)**	94	96	98	81	88	78
`java_api_spring_clean`	`inventory-service`	Java 21 / Spring Boot 3	**78 (B)**	86	82	92	66	74	58
`py_cli_audit_tool`	`logpulse`	Python CLI utility	**77 (B)**	90	80	84	64	68	78
`java_api_spring_messy`	`legacy-billing`	Legacy Java 8/11 Spring	**23 (D)**	29	22	35	14	18	17
`py_api_legacy_messy`	`legacy-order-hub`	Legacy Python Flask API	**20 (D)**	22	24	26	12	18	12
`php_wp_plugin_messy`	`wp_plugin_messy`	Procedural WP plugin	**20 (D)**	24	26	12	18	22	10
`php_legacy_shop_messy`	`legacy_shop_messy`	Procedural PHP 7.4 shop	**15 (D)**	18	19	12	10	18	5

6.3.2 Reference Migration Execution Plans (`based_on` Grade D Audits)

Building directly upon the completed SQLite audit runs of the two lowest-scoring enterprise web services, `pni501` ships two complete reference migration plans demonstrating distinct transformation modes and rollout strategies:

Migration Workflow ID	Prerequisite Audit (`based_on`)	Mode & Rollout Strategy	Target Stack & To-Be Architecture
`java_api_spring_messy_migration`	`java_api_spring_messy` (`legacy-billing` , **23 / D**)	`architectural_restructure` / `phased_pr` (`strict` API, `ask_human`)	Java 21 LTS + Spring Boot 3.4 BOM + PostgreSQL 16 (JPA/Flyway) + Redis 7 in **Clean Architecture** (`domain` / `port` / `usecase` / `adapter`); OpenRewrite codemods isolated from manual cycle cuts (`BillingManager <-> BillingController`).
`php_legacy_shop_messy_migration`	`php_legacy_shop_messy` (`legacy_shop_messy` , **15 / D**)	`cross_stack_replacement` / `strangler_fig` (`strict` API, `ask_human`)	Nginx reverse-proxy **Strangler Fig** routing endpoint-by-endpoint (`/product.php` $\to$ `/` $\to$ `/login.php` $\to$ `/admin_backup.php`) onto PHP 8.3 (`strict_types=1`) + **Modular Laravel 11** (`Domain` / `Actions` / `Http` / `Infrastructure`) + MySQL 8 Eloquent/PDO + Redis 7 cart/sessions, gated by all 19 `.pni/pentests/` DAST regression probes.

6.4 Complete `pni` CLI Quick Reference

All workspace initialization, rule inspection, stateful workflow execution, ad-hoc action debugging, live HTTP penetration testing, Drawlib report compilation, and JSON schema verification operations are exposed under the unified `pni` CLI (`src/pni/cli.py`):

Command	Key Options & Arguments	Primary Purpose
`pni init`	`[--lang/-l python|java|php] [--force/-f]`	Scaffold `.pni/` (`rules/` , `actions/` , `workflows/` , `instructions.md` , and `.pni/cache/.gitignore`).
`pni instructions`	*(none)*	Print the Markdown AI agent bootstrap and stateful execution loop guide to `stdout` .
`pni rules list`	`[--all/-a]`	List active engineering rules and workflow guides with local `.pni/rules/` override status.
`pni rules show`	`<name> [--pretty/-p]`	Output a canonical or overridden rule/guide in clean Markdown (or a Rich panel with `--pretty`).
`pni workflows list`	`[--category/-c audit|migration]`	List project workflows in `.pni/workflows/` with `based_on` links, step counts, targets, and outputs.
`pni workflows show`	`<workflow_id>`	Inspect resolved targets/outputs, normalized primitive steps, and pentest/migration preflight checks.
`pni workflows start`	`<workflow_id> [--supersede]`	Create a stateful run (`wf-...`) in `.pni/cache/workflow_state.db` (`--supersede` cancels prior active runs).
`pni workflows get-action`	`<run_id>`	Execute pending deterministic analyzers automatically and emit the next `act-XX` prompt & JSON schema.
`pni workflows push-action`	`<run_id> <action_id> [input_json] [--file/-f <path>]`	Validate agent JSON input against Pydantic schemas/repo paths, persist to SQLite, and advance state.
`pni workflows run-action`	`<workflow_id> <action> [--base-url <url>] [--force-pentests] [--fail-on-findings]`	Execute a single step/action directly with zero prior SQLite run preparation for rapid iteration or CI.
`pni workflows status`	`[run_id] [-w <wf>] [-s <status>] [--latest] [-n <limit>] [--json]`	Inspect run history or action-by-action step progress (`done/total` , current step) from SQLite.
`pni workflows inspect-action`	`<run_or_workflow> <action> [--agent-input | --deterministic] [--json] [-o <path>]`	Export recorded deterministic JSON facts or pushed agent inputs from SQLite without raw DB queries.
`pni workflows cancel`	`<run_id> | --superseded [-w <workflow_id>]`	Cancel an abandoned `in_progress` run or clean up older superseded runs in SQLite.
`pni pentests init`	`[-w <wf>] [-t <root>] [-s <src>] [-l <lang>] [--force/-f]`	Materialize explicit `Python + httpx` penetration test suites (`conftest.py` , `test_*.py`) in `.pni/pentests/` .

Command	Key Options & Arguments	Primary Purpose
`pni pentests run`	`[-w <wf>] [-t <root>] [-s <src>] [--base-url <url>] [--workflow-id <id>] [--container-engine <eng>] [-f]`	Boot an isolated loopback target container (or probe `--base-url`) and run `.pni/pentests/<target>/`.
`pni report init`	`[target] [--src-dir reports_src] [-l en] [-s google] [--keep-samples] [-f]`	Non-destructively scaffold or refresh `reports_src/<target>_src/` while preserving existing `*.md` chapters.
`pni report build`	`[target] [--src-dir reports_src] [-o reports] [-f pdf|html|markdown|images] [--toc/--no-toc] [--no-cache]`	Run `lint_drawlib_chapters`, compile `reports/<target>.pdf` (default), and verify HTML links when `-f html`.
`pni validate-score`	`[score_path]`	Validate a `*_score.json` file against the Pydantic `ScoreResult` schema.
`pni validate-migration-plan`	`[plan_path]`	Validate a `*_migration_plan.json` file against the Pydantic `MigrationPlanResult` schema.
`pni version`	*(none)*	Print the installed `pni501` package name and version.